МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра квантовых информационных технологий,
практической и прикладной информатики (КИТППИ)

КУРСОВАЯ РАБОТА

по дисциплине «Операционные системы»

Тема курсовой работы: «Планирование и развертывание рабочего места сотрудника и сервера под управлением ОС Linux Ubuntu для тестировщик в компании, осуществляющей услуги по бронированию авиабилетов и отелей»

Студент группы ИВБО-20-24 Дьяков Андрей Геннадьевич _____
(подпись)

Руководитель
курсовой работы к.э.н., доцент Семичастнов М.А. _____
(подпись)

Работа представлена к защите «___»_____2026 г.

Допущен к защите «___»_____2026 г.

Москва 2026 г.

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра квантовых информационных технологий,
практической и прикладной информатики (КИТППИ)

Утверждаю
Заведующий кафедрой КИТППИ
_____ Зуев А.С.
(подпись)
«26» февраля 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы

по дисциплине «Операционные системы»

Студент Дьяков Андрей Геннадьевич Группа ИВБО-20-24

Тема «Планирование и развертывание рабочего места сотрудника и сервера под управлением ОС Linux Ubuntu для тестировщик в компании, осуществляющей услуги по бронированию авиабилетов и отелей»

Исходные данные:

	Параметр предприятия	**Значение**
1	Дистрибутив, используемый в составе АРМ	Ubuntu Desktop
2	Дистрибутив, используемый на сервере	Ubuntu Server

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. **Анализ требований и проектирование инфраструктуры**
   Выполнить спецификацию бизнес-процесса, в котором участвует сотрудник. Описать функциональные обязанности, а также декомпозицию процесса до уровня функциональных действий, выполняемых с использованием АРМ.
   Специфицировать нефункциональные требования к производительности и удобству использования. Привести диаграмму, отражающую место АРМ и сервера в общем контуре деятельности организации. Возможно дополнение спецификации диаграммой потоков данных.

2. **Определение классов прикладного и инструментального программного обеспечения и обоснование выбора дистрибутива Linux**

   Определить классы прикладного и инструментального программного обеспечения, необходимые для выполнения должностных задач сотрудника. Выполнить сравнительный анализ проприетарных (в том числе импортных), свободных и отечественных альтернатив.

   Рассмотреть выбор конкретных версий ОС для рабочей станции и сервера (с учётом их наличия в реестре отечественного ПО, требований безопасности, совместимости с оборудованием).

   Спецификация прикладного и инструментального программного обеспечения должна содержать карту покрытия функциональных задач сотрудника функциональными возможностями программного обеспечения, а также сравнительный анализ различных версий и типов ПО. Должна быть приведена итоговая спецификация прикладного и инструментального ПО с указанием применяемых продуктов и их версий.

3. **Проектирование аппаратной инфраструктуры и сетевой топологии**

   Спецификация проекта должна содержать схему развертывания; должна быть приведена итоговая спецификация системного ПО (операционных систем) для рабочего места сотрудника и сервера с указанием применяемых дистрибутивов и их версий.

   Разработать спецификацию аппаратного обеспечения для АРМ и сервера, подтверждённую совместимостью с выбранными дистрибутивами Linux. Привести схему развёртывания программного обеспечения: распределение компонентов (ОС, драйверы, прикладные пакеты, сервисы) между рабочей станцией и сервером, способы взаимодействия (протоколы, порты, сетевые экраны).

   Для серверной части инфраструктуры необходимо выполнить проектирование следующих сетевых служб: DHCP сервера, DNS, службы каталогов, систем конфигурационного управления и мониторинга, пакетного менеджера.

4. **Описание развертывания и тестирования функционального прототипа данной конфигурации**

   В данном разделе приводится описание функционального прототипа, полученного в результате тестового развертывания системного и прикладного ПО на рабочее место сотрудника и сервер. Перед развертыванием приводится описание ограничений функционального прототипа и состав применяемого аппаратного обеспечения и средств виртуализации. Описать процесс установки ОС на АРМ и сервер с обоснованием параметров (разметка диска, настройка системы). Привести детальные конфигурации системных и сетевых настроек (сетевые интерфейсы и службы, SSH, права доступа, пакетные репозитории). Выполнить установку и настройку прикладного ПО на АРМ пользователя и сервере в соответствии со спецификацией из п. 2. Зафиксировать основные конфигурационные файлы (листинги) и скриншоты ключевых этапов. Также необходимо выполнить оценку потребляемых ресурсов и, по возможности, развернуть службы мониторинга.

   Обязательный графический материал: диаграмма развёртывания для функционального прототипа, скриншоты рабочих окружений и серверных консолей, таблицы результатов тестирования, графики утилизации ресурсов (при необходимости).

Срок представления к защите курсовой работы: **до** «22» мая 2026 г.

Задание на курсовую работу выдал _____ Семичастнов М.А.

Подпись руководителя *(ФИО руководителя)*

«26» февраля 2026 г.

Задание на курсовую работу получил _____ Дьяков А.Г.

Подпись обучающегося *(ФИО обучающегося)*

«26» февраля 2026 г.

АННОТАЦИЯ

Данная работа включает в себя 56 рисунков, 14 таблиц, 1 листинг. Количество страниц в работе — 57.

СОДЕРЖАНИЕ

АННОТАЦИЯ 1

СОДЕРЖАНИЕ 5

ВВЕДЕНИЕ 7

1 Анализ требований и проектирование инфраструктуры 9

1.1 Описание бизнес-процесса 9

1.2 Функциональные обязанности тестировщика 9

1.3 Декомпозиция бизнес-процесса 10

1.4 Нефункциональные требования 11

1.5 Диаграмма взаимодействия компонентов инфраструктуры 12

2 Определение классов прикладного и инструментального программного обеспечения 14

2.1 Классы программного обеспечения 14

2.2 Набор ПО 14

2.3 Карта покрытия задач ПО 15

2.4 Сравнительный анализ ПО 16

2.5 Выбор дистрибутива Linux 19

2.6 Итоговая спецификация ПО 19

3 Проектирование аппаратной инфраструктуры и сетевой топологии 21

3.1 Итоговая спецификация системного программного обеспечения 21

3.2 Спецификация аппаратного обеспечения 21

3.3 Обоснование совместимости оборудования с дистрибутивами Linux 23

3.4 Проектирование сетевых служб серверной инфраструктуры 23

3.5 Схема развёртывания программного обеспечения и настройка сетевого экрана 25
4 Описание развертывания и тестирования функционального прототипа данной конфигурации 27
4.1 Ограничения прототипа и средства виртуализации 27
4.2 Установка и базовая настройка операционных систем 28
4.3 Настройка сетевых интерфейсов и базовых служб 29
4.4 Развертывание серверного прикладного ПО 38
4.5 Развертывание прикладного ПО на АРМ тестировщика 44
4.6 Развертывание системы управления базами данных 46
4.7 Тестирование функционального прототипа инфраструктуры 50
4.7.1 Функциональное тестирование серверных компонентов 50
4.7.2 Тестирование API тестируемого сервиса 52
4.7.3 Тестирование производительности и нагрузки 53
4.7.4 Результаты тестирования 54
4.7.5 Итоговая оценка функционального прототипа 54
ЗАКЛЮЧЕНИЕ 56
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 59

ВВЕДЕНИЕ

Актуальность темы. В условиях стремительного развития информационных технологий и перехода большинства бизнес-процессов в цифровую среду обеспечение качества программного обеспечения становится критически важной задачей. QA-инженеры (специалисты по обеспечению качества) играют ключевую роль в жизненном цикле разработки, выявляя дефекты до выпуска продукта конечным пользователям. Эффективность работы тестировщика напрямую зависит от качества используемой инфраструктуры: автоматизированного рабочего места, серверных сервисов для управления проектами, отслеживания дефектов и хранения тестовых данных.

Развертывание полноценной инфраструктуры для QA-тестировщика требует тщательного подбора программного обеспечения, проектирования сетевой топологии, настройки средств автоматизации и обеспечения безопасности. При этом современные тенденции в IT-индустрии диктуют необходимость использования свободного программного обеспечения с открытым исходным кодом, что позволяет избежать лицензионных ограничений, обеспечить независимость от вендоров и адаптировать решения под конкретные потребности организации.

Особую значимость приобретает вопрос выбора операционной системы. Семейство дистрибутивов Linux, в частности Ubuntu, зарекомендовало себя как надежная и безопасная платформа как для серверных, так и для клиентских решений. Использование контейнеризации (Docker), систем управления конфигурациями (Ansible) и современных средств мониторинга (Netdata) позволяет создавать масштабируемые и легко поддерживаемые инфраструктуры.

Цель работы — проектирование и развертывание функционального прототипа инфраструктуры для QA-тестировщика на базе операционных систем семейства Linux с использованием средств виртуализации, контейнеризации и автоматизации.

Объектом исследования является процесс организации инфраструктуры для обеспечения деятельности QA-тестировщика.

Предметом исследования выступают методы и средства проектирования, развертывания и администрирования серверной и клиентской инфраструктуры на базе операционных систем Linux.

Методы исследования. В работе использованы методы системного анализа для декомпозиции бизнес-процесса, методы сравнительного анализа для обоснования выбора программного обеспечения, а также практические методы администрирования операционных систем Linux, настройки сетевых служб, контейнеризации приложений и автоматизации задач управления конфигурациями.

Практическая значимость работы заключается в создании готового к использованию функционального прототипа инфраструктуры QA-тестировщика, который может быть применен в учебных целях, а также послужить основой для развертывания полноценной рабочей среды в организации. Все использованные программные решения являются свободными и не требуют приобретения коммерческих лицензий.

1 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ ИНФРАСТРУКТУРЫ

1.1 Описание бизнес-процесса

Рассматриваемый бизнес-процесс:

Тестирование системы бронирования авиабилетов и отелей.

Цель процесса:

Обеспечение качества программного продукта (веб/мобильного сервиса бронирования), выявление дефектов и подтверждение соответствия требованиям.

Участник процесса:

QA-тестировщик

1.2 Функциональные обязанности тестировщика

Должностные обязанности QA-тестировщика в рамках данного бизнес-процесса охватывают полный цикл проверки программного обеспечения. К основным функциональным обязанностям сотрудника относятся:

1. Анализ технической документации и пользовательских сценариев.
2. Разработка тестовой документации, включая тест-планы и тест-кейсы.
3. Проведение ручного и автоматизированного функционального тестирования.
4. Выявление, локализация и документирование дефектов в специализированных системах отслеживания.
5. Взаимодействие с командой разработки для уточнения требований и контроля исправления ошибок.
6. Проведение регрессионного тестирования после внесения изменений в код.

7. Формирование итоговых отчетов о качестве программного продукта и оценка готовности к релизу.

1.3 Декомпозиция бизнес-процесса

Для детального проектирования инфраструктуры бизнес-процесс декомпозируется на три уровня детализации.

Уровень 1 (Общий процесс) включает пять основных этапов: получение требований, подготовка к тестированию, непосредственное выполнение тестирования, обработка выявленных дефектов и подготовка итоговой отчетности.

Уровень 2 (Детализация этапов) раскрывает содержание каждого этапа. Например, этап подготовки включает создание тест-планов, разработку тест-кейсов и подготовку тестовых данных, а этап выполнения тестирования подразделяется на функциональное, UI/UX и API-тестирование.

Уровень 3 (Функциональные действия на АРМ) описывает конкретные операции, выполняемые тестировщиком с использованием программного обеспечения рабочей станции. К ним относятся: запуск тестовой среды; ввод тестовых данных для поиска рейсов или отелей; визуальная и программная проверка результатов (цены, доступность мест); регистрация и приоритезация дефектов в баг-трекинг системе; анализ системных логов с использованием SSH-клиентов; а также формирование и экспорт отчетов о тестировании.

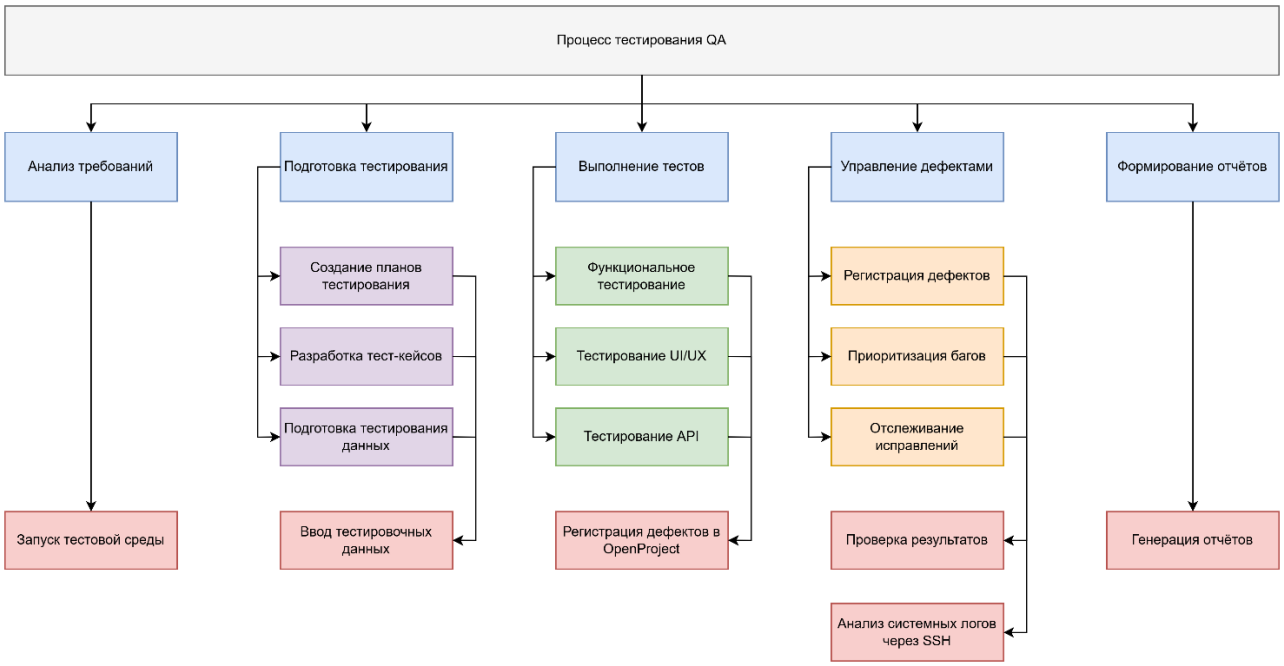


Рисунок 1.1 — Иерархическая декомпозиция бизнес-процесса тестирования на три уровня детализации

1.4 Нефункциональные требования

Для обеспечения эффективной работы QA-тестировщика к автоматизированному рабочему месту и серверной инфраструктуре предъявляется ряд нефункциональных требований.

В части производительности время отклика пользовательского интерфейса при загрузке страниц не должно превышать 2–3 секунд, а обработка запросов к базе данных при бронировании — до 5 секунд.

Требования к надежности подразумевают высокую отказоустойчивость серверных компонентов и гарантированную сохранность тестовых данных и истории дефектов.

С точки зрения удобства использования (usability), интерфейс прикладного ПО должен быть интуитивно понятным, обеспечивать быстрый доступ к часто используемым инструментам и поддерживать удобную навигацию по логам и отчетам.

Требования информационной безопасности включают строгое разграничение прав доступа к системе, использование защищенных протоколов

(SSH, HTTPS) для взаимодействия с сервером и соблюдение политик защиты персональных данных, обрабатываемых в тестовой среде.

1.5 Диаграмма взаимодействия компонентов инфраструктуры

Для визуализации места автоматизированного рабочего места (АРМ) и сервера в общем информационном контуре организации разработана структурная диаграмма взаимодействия компонентов (Рисунок 1.1). Диаграмма отражает основные потоки данных между QA-тестировщиком, его рабочей станцией и серверной инфраструктурой.

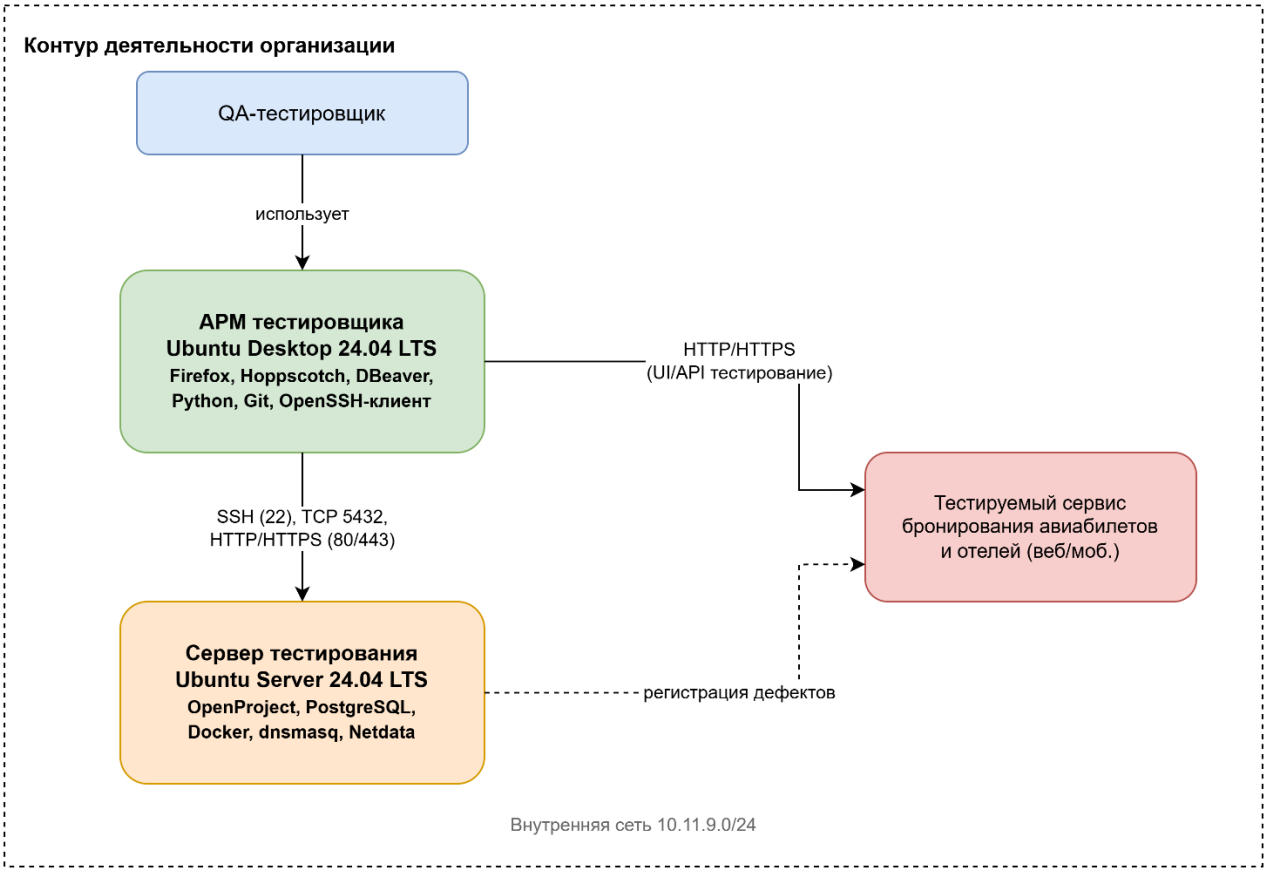


Рисунок 1.2 — Диаграмма взаимодействия компонентов инфраструктуры

Для визуализации места автоматизированного рабочего места (АРМ) и сервера в общем информационном контуре организации разработана структурная диаграмма взаимодействия компонентов (Рисунок 1.1). Диаграмма

отражает основные потоки данных между QA-тестировщиком, его рабочей станцией и серверной инфраструктурой.

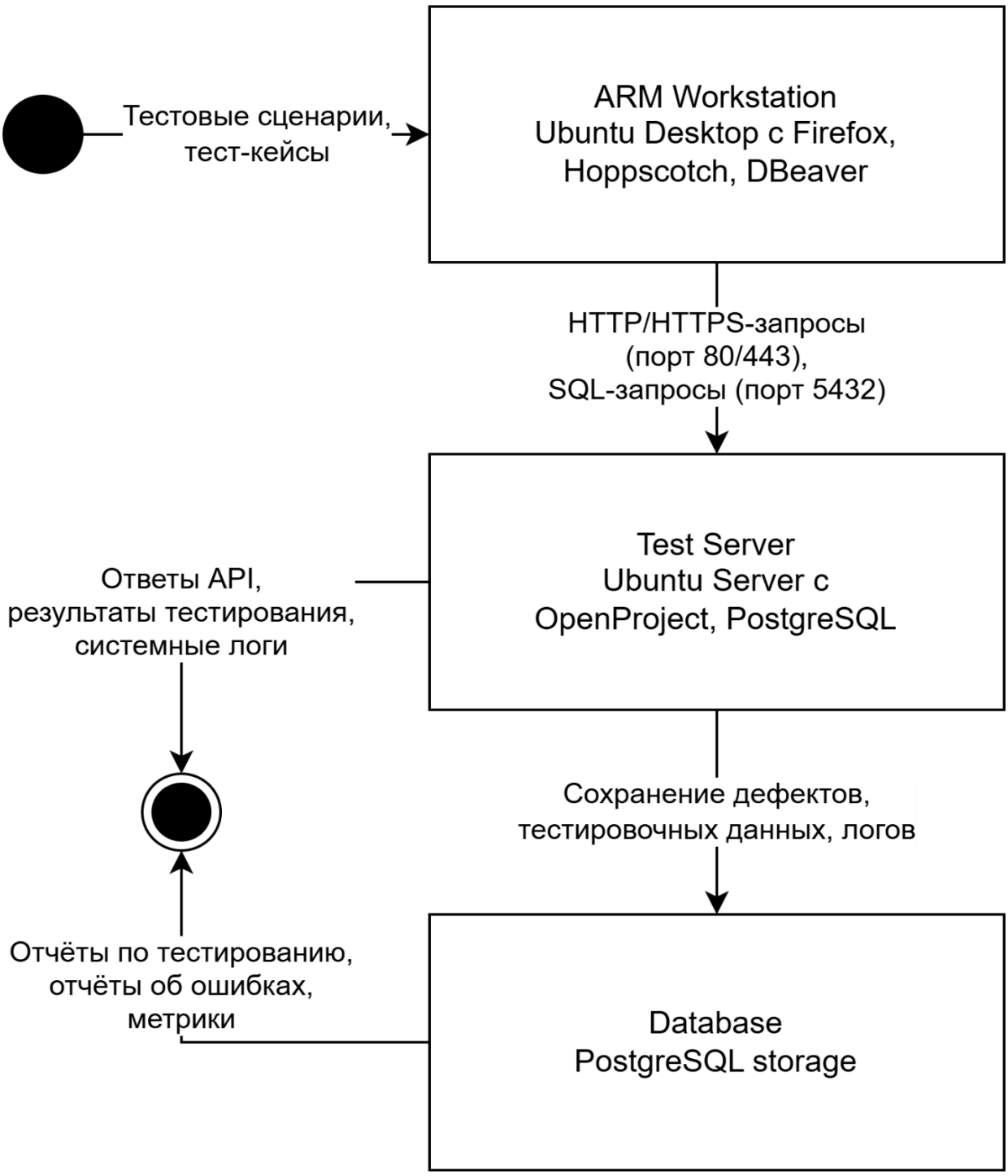


Рисунок 1.3 — Диаграмма потоков данных между компонентами инфраструктуры тестирования

2 ОПРЕДЕЛЕНИЕ КЛАССОВ ПРИКЛАДНОГО И ИНСТРУМЕНТАЛЬНОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Классы программного обеспечения

Для обеспечения полноценной деятельности QA-тестировщика в рамках рассматриваемого бизнес-процесса необходимо разделить используемое программное обеспечение на два основных класса: прикладное и инструментальное.

Прикладное программное обеспечение непосредственно используется для решения конкретных профессиональных задач сотрудника, таких как взаимодействие с пользовательским интерфейсом тестируемого продукта, проверка программных интерфейсов, регистрация выявленных дефектов и анализ системных логов.

В свою очередь, инструментальное программное обеспечение предоставляет базовую среду для разработки, автоматизации рутинных процессов, управления версиями исходного кода и обеспечения безопасного удаленного доступа к серверной инфраструктуре. Такое разделение позволяет четко определить архитектурные границы между клиентской рабочей станцией и серверным оборудованием.

2.2 Набор ПО

Формирование итогового набора программного обеспечения базируется на принципах открытости, кроссплатформенности и отсутствия необходимости в приобретении коммерческих лицензий.

На автоматизированном рабочем месте тестировщика устанавливается операционная система Ubuntu Desktop, которая дополняется прикладными инструментами для визуального и программного тестирования, а также

инструментальными средствами для написания автотестов и работы с репозиториями.

Серверная часть инфраструктуры, функционирующая под управлением Ubuntu Server, берет на себя роль хостинга для системы управления проектами, реляционной базы данных и среды контейнеризации, что обеспечивает централизованное хранение артефактов тестирования и изоляцию сервисов.

Таблица 2.1 — АРМ тестировщика (Ubuntu Desktop)

Класс	ПО	Назначение
Прикладное	Firefox	UI тестирование
Прикладное	Hoppscotch	API тестирование
Инструментальное	Git	Контроль версий
Инструментальное	Python	Автотесты
Инструментальное	OpenSSH	Доступ к серверу
Инструментальное	DBeaver Community	Клиент для СУБД

Таблица 2.2 — Сервер (Ubuntu Server)

Класс	ПО	Назначение
Прикладное	OpenProject	Баг-трекинг
Прикладное	PostgreSQL	База данных
Инструментальное	Docker	Развертывание
Инструментальное	Netdata	Развертывание

2.3 Карта покрытия задач ПО

Для подтверждения достаточности выбранного набора программного обеспечения была разработана карта покрытия функциональных задач. Данная матрица демонстрирует прямую корреляцию между должностными обязанностями QA-тестировщика, описанными в первой части курсовой работы, и конкретными программными продуктами, выделенными для их выполнения.

Таблица 2.3 — Карта покрытия задач ПО

Задача тестировщика	ПО
Тестирование UI	Firefox
Тестирование API	Hoppscotch
Регистрация багов	OpenProject
Проверка логов	SSH + Linux
Автотесты	Python
Работа с репозиторием	Git

2.4 Сравнительный анализ ПО

Обоснование выбора конкретных программных продуктов требует проведения сравнительного анализа существующих на рынке решений с учетом их лицензионной чистоты, доступности на территории Российской Федерации и функционального богатства

Таблица 2.4 — Bug Tracking системы

Тип	ПО	Особенности
Проприетарное	Jira	Мощная, но платная
Свободное	OpenProject	Полный аналог Jira
Свободное	Redmine	Лёгкое и бесплатное
Свободное	Kaiten	Облачные SaaS-решения, есть в реестре РФ

Анализ систем отслеживания дефектов показывает, что проприетарное решение Jira, являясь отраслевым стандартом, обладает высокой стоимостью лицензирования и сложностью первоначальной настройки.

В качестве альтернативы рассматривалось свободное программное обеспечение OpenProject, которое предоставляет сопоставимый функционал для управления проектами и регистрации дефектов.

Несмотря на наличие на российском рынке платформ для управления задачами, таких как Kaiten, они представляют собой облачные SaaS-решения и не подходят для локального развертывания в изолированном контуре предприятия.

В связи с этим оптимальным выбором для функционального прототипа становится OpenProject, позволяющий полностью контролировать данные на собственном сервере без лицензионных отчислений и с минимальными затратами на первоначальную настройку.

Таблица 2.5 — Инструменты API-тестирования

Тип	ПО	Особенности
Проприетарное	Postman	Широко распространенное коммерческое решение, но с закрытым исходным кодом
Свободное	Hoppscotch	Открытый код, легковесный, локальный хостинг
Свободное	curl	Консольный CLI-инструмент для быстрой отладки

В сегменте инструментов для тестирования программных интерфейсов проприетарный Postman долгое время оставался стандартом, однако его политика лицензирования и ориентация на облачные вычисления делают его менее предпочтительным для локальных сред.

Свободный инструмент Hoppscotch предоставляет аналогичный графический интерфейс, имеет полностью открытый исходный код и поддерживает локальное развертывание, что делает его идеальной бесплатной альтернативой.

Консольная утилита curl при этом сохраняется в арсенале тестировщика исключительно как вспомогательное средство для быстрой отладки запросов.

Таблица 2.6 — Браузер

Тип	ПО	Особенности
Свободное	Firefox	Open-source, кроссплатформенный
Проприетарное	Chrome	Закрытая телеметрия, привязка к экосистеме
Свободное	Yandex Browser	Есть в реестре РФ, встроенные средства защиты

При выборе веб-обозревателя для тестирования пользовательских интерфейсов проприетарный Google Chrome уступает свободному Mozilla Firefox по причине закрытости телеметрии.

Отечественный Яндекс Браузер обладает встроенными средствами защиты и входит в реестр российского ПО, однако содержит проприетарные надстройки.

Таким образом, Mozilla Firefox является наиболее сбалансированным решением, сочетающим открытость, отсутствие телеметрии и строгое соблюдение стандартов веб-разработки.

Таблица 2.7 — Контроль версий

Тип	ПО	Особенности
Свободное	Git	Абсолютный отраслевой стандарт

В области систем контроля версий безальтернативным отраслевым стандартом является свободная система Git, обеспечивающая распределенное хранение кода и полную совместимость со всеми современными инструментами непрерывной интеграции и доставки.

Таблица 2.8 — Клиент для СУБД

Тип	ПО	Особенности
Проприетарное	DataGrip	Мощный коммерческий инструмент от JetBrains, требует лицензии
Свободное	DBeaver Community	Универсальный клиент с открытым кодом, поддержка PostgreSQL из коробки
Свободное	pgAdmin 4	Специализированный клиент для PostgreSQL, веб-интерфейс

Анализ клиентских приложений для взаимодействия с системами управления базами данных показывает, что проприетарный DataGrip обладает широким функционалом и удобной средой разработки, однако требует приобретения коммерческой лицензии, что противоречит принципу бесплатности прототипа.

Специализированный клиент pgAdmin 4, несмотря на полную совместимость с PostgreSQL, ориентирован преимущественно на администрирование базы данных и имеет более сложный интерфейс.

В связи с этим оптимальным выбором становится свободный клиент DBeaver Community, который предоставляет интуитивно понятный графический интерфейс для выполнения SQL-запросов, визуализации схем данных, импорта и экспорта информации, полностью покрывая потребности QA-тестировщика при тестировании без каких-либо финансовых затрат.

2.5 Выбор дистрибутива Linux

Выбор дистрибутива операционной системы Linux для автоматизированного рабочего места и сервера продиктован требованиями к производительности, стабильности и совместимости с необходимым стеком технологий.

Для рабочей станции QA- тестировщика выбрана операционная система Ubuntu Desktop версии 24.04 LTS с графическим окружением GNOME. Данный дистрибутив обеспечивает интуитивно понятный пользовательский интерфейс, необходимый для комфортной работы с инструментами тестирования (веб-браузером Firefox, клиентом DBeaver, средой разработки). Версия с длительной поддержкой (LTS) гарантирует получение обновлений безопасности и стабильность работы на протяжении всего жизненного цикла проекта. Полная совместимость с единой пакетной базой Ubuntu обеспечивает беспроблемную установку всех необходимых QA-инструментов через пакетный менеджер APT.

Для серверной части инфраструктуры применяется Ubuntu Server 24.04 LTS. Данный дистрибутив обладает безупречной репутацией в корпоративном секторе, обеспечивая высочайший уровень стабильности и безопасности при минимальных требованиях к аппаратным ресурсам. Отсутствие графического интерфейса по умолчанию позволяет снизить накладные расходы на системные ресурсы, высвобождая их для функционирования контейнеризированных сервисов (OpenProject, PostgreSQL). Наличие обширной базы знаний, регулярных патчей безопасности и нативная поддержка современных версий Docker и PostgreSQL делают Ubuntu Server оптимальной платформой для хостинга серверных компонентов инфраструктуры.

2.6 Итоговая спецификация ПО

Итоговая спецификация программного обеспечения фиксирует конкретные версии продуктов, выбранные для развертывания функционального прототипа. Указанные версии являются стабильными, протестированными на

совместимость друг с другом и поддерживают все заявленные функциональные требования.

Таблица 2.9 — Итоговая спецификация ПО

ПО	Версия
Ubuntu Desktop	24.04 LTS
Ubuntu Server	24.04 LTS
Firefox	147.0.3
Hoppscotch	26.5.0
Git	2.43.0
Python	3.12.3
OpenSSH	3.0.13
OpenProject	14.6.3
PostgreSQL	16.14
Docker	29.6.0
DBeaver Community	26.1.0
Netdata	2.10.0

3 ПРОЕКТИРОВАНИЕ АППАРАТНОЙ ИНФРАСТРУКТУРЫ И СЕТЕВОЙ ТОПОЛОГИИ

3.1 Итоговая спецификация системного программного обеспечения

В соответствии с требованиями задания и результатами выбора, выполненными во втором разделе, для функционирования автоматизированного рабочего места и серверной инфраструктуры применяются дистрибутивы семейства Ubuntu. Использование единой пакетной базы и ядра Linux обеспечивает высокую степень совместимости компонентов и унификацию процессов администрирования.

Выбор версий с длительной поддержкой (LTS) гарантирует стабильность работы инфраструктуры на протяжении всего жизненного цикла разработки и тестирования программного продукта. Итоговая спецификация системного программного обеспечения представлена в Таблице 3.1.

Таблица 3.1 — Итоговая спецификация системного программного обеспечения

Компонент инфраструктуры	Дистрибутив операционной системы	Версия	Архитектура
АРМ тестировщика	Ubuntu Desktop	24.04 LTS	amd64 (x86_64)
Сервер тестирования	Ubuntu Server	24.04 LTS	amd64 (x86_64)

3.2 Спецификация аппаратного обеспечения

Проектирование аппаратной инфраструктуры базируется на использовании среды виртуализации, что подтверждается практическим опытом развертывания учебных стендов в Oracle VM VirtualBox. Выделение вычислительных ресурсов осуществляется с учетом специфики решаемых задач: для АРМ тестировщика требуется достаточный объем оперативной памяти для запуска легковесного графического окружения XFCE, веб-браузера с открытыми

вкладками и средств разработки автотестов. Спецификация аппаратных ресурсов АРМ представлена в Таблице 3.2.

Таблица 3.2 — Спецификация аппаратных ресурсов АРМ тестировщика

Компонент	Характеристики	Обоснование выбора
Процессор (vCPU)	1 ядра	Достаточно для графического интерфейса и локальных скриптов Python
Оперативная память	4 ГБ	Оптимально для Ubuntu Desktop и одновременной работы браузера с DBeaver
Накопитель	20 ГБ SSD	Хранение ОС, инструментов разработки и локальных репозиториев
Сетевой адаптер	1 Гбит/с (Virtio)	Обеспечивает быстрый обмен данными с сервером по SSH и HTTP

Серверная часть инфраструктуры требует повышенных ресурсов, в частности оперативной памяти, для корректного функционирования стека OpenProject, СУБД PostgreSQL и контейнеров мониторинга без деградации производительности при проведении тестирования. Спецификация аппаратных ресурсов сервера представлена в Таблице 3.3.

Таблица 3.3 — Спецификация аппаратных ресурсов сервера тестирования

Компонент	Характеристики	Обоснование выбора
Процессор (vCPU)	1 ядра	Достаточно для последовательного выполнения сервисов в рамках учебного прототипа; для продуктивной среды рекомендуется увеличение до 4 vCPU
Оперативная память	8 ГБ	Критично для работы Java/Ruby-стека OpenProject и Prometheus
Накопитель	40 ГБ SSD	Хранение баз данных, логов, Docker-образов и бэкапов
Сетевой адаптер	1 Гбит/с (Virtio)	Высокая пропускная способность для API-запросов и передачи логов

3.3 Обоснование совместимости оборудования с дистрибутивами Linux

Выбранная аппаратная и виртуальная платформа обладает полной нативной поддержкой со стороны ядра Linux. Архитектура x86_64 (amd64) является основной целевой платформой для дистрибутивов Ubuntu и Ubuntu Desktop, что гарантирует отсутствие проблем с совместимостью на уровне машинных инструкций. Виртуальные сетевые адаптеры и дисковые контроллеры (virtio), используемые в среде виртуализации, поддерживаются открытыми драйверами, входящими в состав стандартного ядра Linux, что исключает необходимость установки проприетарных прошивок.

Как отмечается в профильной технической литературе, ядро Linux обладает встроенной поддержкой стандартных сетевых и дисковых подсистем, что обеспечивает стабильную работу сетевых служб и файловых систем ext4 без привлечения сторонних закрытых модулей. Отсутствие необходимости в использовании специализированного графического ускорителя для задач UI и API тестирования позволяет полностью отказаться от проприетарных видеодрайверов, сохраняя концепцию стопроцентно свободного программного обеспечения на всех уровнях стека. Таким образом, выбранное оборудование и виртуальные ресурсы полностью совместимы с выбранными дистрибутивами и не требуют дополнительных затрат на лицензирование закрытых драйверов.

3.4 Проектирование сетевых служб серверной инфраструктуры

Выбор программного обеспечения для сетевых служб базируется на принципах открытости, бесплатности и простоты развертывания.

Для автоматического распределения IP-адресов и разрешения доменных имен применяется dnsmasq, объединяющий функциональность DHCP- и DNS-сервера в одном пакете с единым конфигурационным файлом /etc/dnsmasq.conf.

Это решение полностью покрывает потребности инфраструктуры из одного АРМ и одного сервера.

Для автоматизации задач администрирования применяется Ansible — безагентная система конфигурационного управления, работающая по SSH. Ansible написан на Python, что унифицирует среду с инструментарием АРМ тестировщика.

Мониторинг серверных ресурсов обеспечивается системой Netdata, устанавливаемой одной командой и предоставляющей веб-интерфейс с графиками реального времени без дополнительной настройки.

Базовым инструментом управления программным обеспечением является пакетный менеджер APT в стандартной конфигурации.

Служба каталогов (LDAP/FreeIPA) в данной конфигурации не разворачивается, поскольку инфраструктура включает единственную рабочую станцию и единственный сервер с одним пользователем; централизованное управление учётными записями избыточно и не несёт практической пользы при выбранном масштабе. Аутентификация реализуется локальными учётными записями Linux

Итоговая спецификация сетевых служб представлена в Таблице 3.4.

Таблица 3.4 — Спецификация сетевых служб серверной инфраструктуры

Служба	Программное обеспечение	Назначение в инфраструктуре
DHCP	dnsmasq	Автоматическая выдача IP-адресов клиентским машинам
DNS	dnsmasq	Разрешение внутренних доменных имен (openproject.local)
Конфигурационное управление	Ansible	Автоматизация настроек сервера через SSH
Мониторинг	Netdata	Сбор метрик ресурсов и визуализация нагрузки
Пакетный менеджер	APT	Установка и обновление программного обеспечения

3.5 Схема развёртывания программного обеспечения и настройка сетевого экрана

Распределение компонентов построено по клиент-серверной архитектуре. На АРМ тестировщика (Ubuntu Desktop 24.04 LTS) размещаются клиентские средства: Firefox, Hoppscotch, DBeaver Community, Python, Git, OpenSSH. Серверная часть (Ubuntu Server 24.04 LTS) содержит OpenProject, PostgreSQL, Docker, dnsmasq и Netdata.

Взаимодействие осуществляется по стандартным протоколам: HTTP/HTTPS (порты 80/443) для веб-интерфейсов OpenProject и Netdata, TCP 5432 для подключения к PostgreSQL, SSH (порт 22) для удаленного администрирования и Ansible, DNS (порт 53) и DHCP (порт 67) для службы dnsmasq, порт 19999 для Netdata. Схема развертывания представлена на Рисунке 3.1.

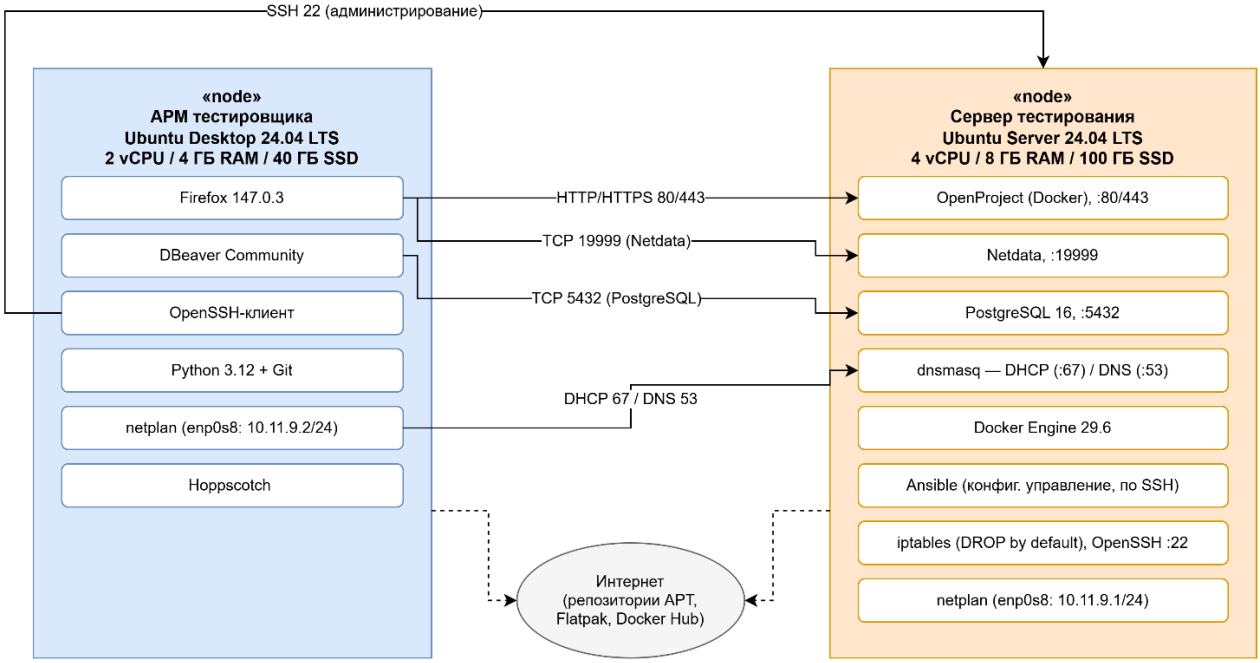


Рисунок 3.1 — Схема развёртывания программного обеспечения

Настройка сетевых интерфейсов осуществляется через netplan (конфигурационные файлы в /etc/netplan/). Серверу назначается статический IP-адрес для стабильности сетевых взаимодействий.

Для безопасности применяется iptables с политикой «запрещено всё, что не разрешено явно» (iptables -P INPUT DROP). Разрешены только необходимые порты: 22 (SSH), 80/443 (HTTP/HTTPS), 5432 (PostgreSQL), 53 (DNS), 19999 (Netdata), а также установленные соединения (iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT).

4 ОПИСАНИЕ РАЗВЕРТЫВАНИЯ И ТЕСТИРОВАНИЯ ФУНКЦИОНАЛЬНОГО ПРОТОТИПА ДАННОЙ КОНФИГУРАЦИИ

4.1 Ограничения прототипа и средства виртуализации

Развертывание функционального прототипа осуществляется в среде виртуализации Oracle VM VirtualBox версии 7.x, что обусловлено учебным характером работы и необходимостью изоляции тестового контура. Виртуализация позволяет гибко распределять аппаратные ресурсы, быстро создавать снимки состояния машин и моделировать сетевые топологии без использования физического оборудования.

Функциональный прототип имеет ряд ограничений: отсутствует резервирование серверных компонентов, не реализована кластеризация базы данных PostgreSQL и системы OpenProject, сетевая инфраструктура ограничена одним внутренним сегментом без организации DMZ. Данные ограничения допустимы для учебного функционального прототипа.

АРМ тестировщика на базе Ubuntu Desktop 24.04 LTS выделен 1 виртуальный процессор, 4 ГБ оперативной памяти и 20 ГБ дискового пространства. Сервер тестирования на базе Ubuntu Server 24.04 LTS оснащен 1 виртуальным процессором, 8 ГБ оперативной памяти и 40 ГБ дискового пространства. Суммарное выделение виртуальных процессоров (2 vCPU) соответствует количеству физических ядер хостовой машины. Использование динамических виртуальных дисков формата VDI позволяет экономить физическое дисковое пространство.

Сетевая топология построена по принципу dual-homed архитектуры. Каждая виртуальная машина оснащена двумя сетевыми адаптерами: первый работает в режиме NAT для доступа к сети Интернет, второй подключен к внутренней сети VirtualBox (Internal Network) с именем intnet для изолированного взаимодействия между АРМ и сервером по статическим IP-

адресам. Общая схема размещения виртуальных машин и их сетевых адаптеров представлена на Рисунке 4.1.

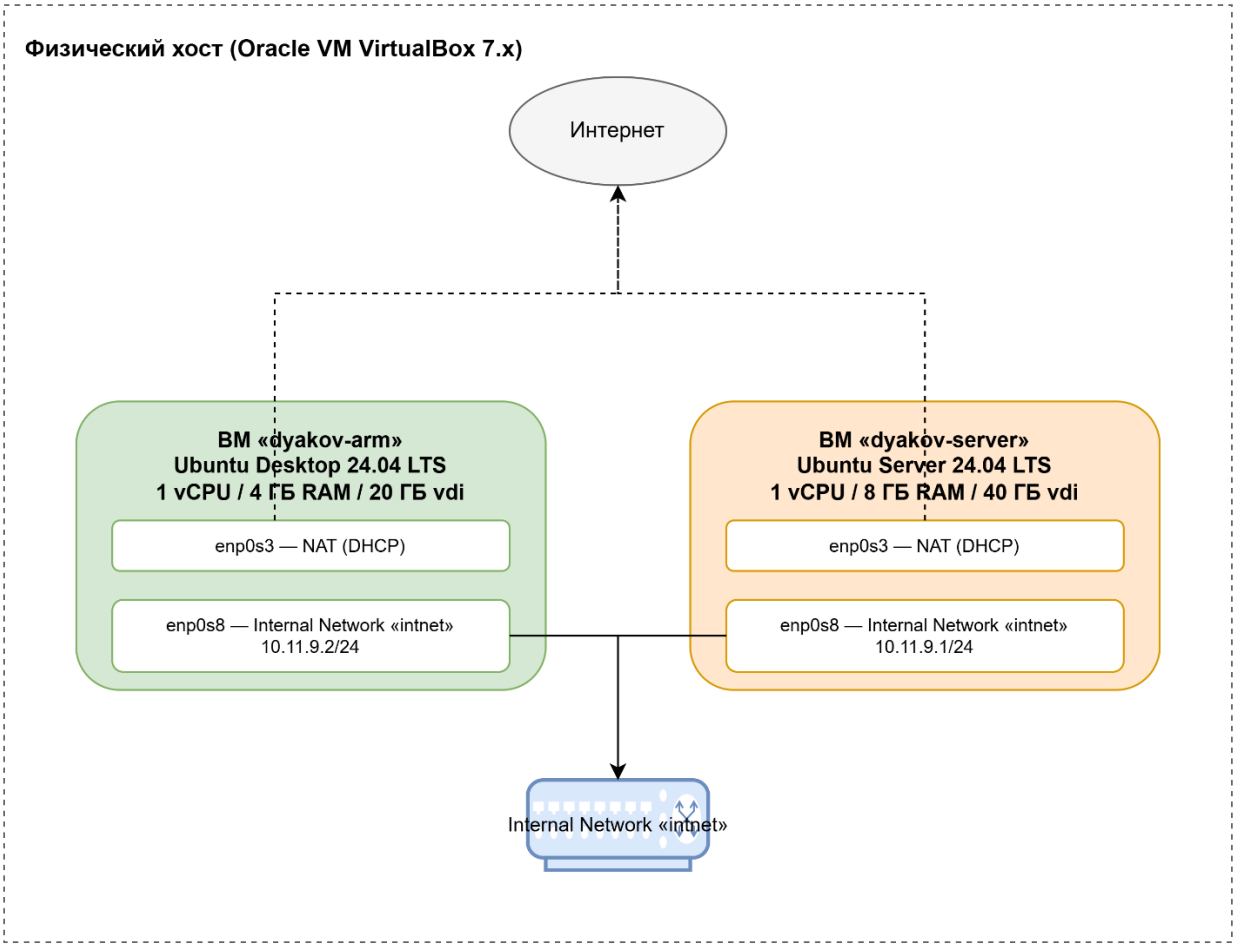


Рисунок 4.1 — Диаграмма развертывания прототипа

4.2 Установка и базовая настройка операционных систем

Развертывание функционального прототипа начинается с установки операционных систем на виртуальные машины. Для АРМ тестировщика используется дистрибутив Ubuntu Desktop 24.04 LTS с графическим окружением GNOME. Для сервера применяется Ubuntu Server 24.04 LTS с консольным установщиком. Обе системы разворачиваются в среде Oracle VM VirtualBox с использованием динамических виртуальных дисков формата VDI.

Разметка дискового пространства выполнена по упрощенной схеме. На АРМ выделен корневой раздел / размером 18 ГБ и раздел подкачки swap объемом

2 ГБ. На сервере создан корневой раздел / размером 36 ГБ и раздел подкачки swap объемом 4 ГБ.

Для обоих дистрибутивов создан локальный пользователь dyakov, входящий в группу sudo. Настройка беспарольного доступа к sudo выполнена путем добавления директивы в файл /etc/sudoers.d/dyakov — данная настройка выполнена идентично на сервере тестирования (Рисунок 4.2) и на АРМ тестировщика (Рисунок 4.3).

```
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "@include" dire
dyakov ALL=(ALL) NOPASSWD: ALL
@includedir /etc/sudoers.d
```

Рисунок 4.2 — Настройка беспарольного sudo на АРМ тестировщика

```
dyakov ALL=(ALL) NOPASSWD: ALL
@includedir /etc/sudoers.d
```

Рисунок 4.3 — Настройка беспарольного sudo на сервере тестирования

После завершения установки на обеих машинах выполнены базовые операции: обновление пакетной базы и установка минимального набора утилит (curl, wget, net-tools, htop), а также настройка имен хостов (dyakov-arm и dyakov-server). Результат установки утилит на АРМ показан на Рисунке 4.4, на сервере — на Рисунке 4.5.

```
dyakov@dyakov-arm:~$ sudo apt install -y curl wget net-tools htop
```

Рисунок 4.4 — Установка минимального набора утилит на АРМ тестировщика

```
dyakov@dyakov-server:~$ sudo apt install -y curl wget net-tools htop_
```

Рисунок 4.5 — Установка минимального набора утилит на сервере тестирования

4.3 Настройка сетевых интерфейсов и базовых служб

Настройка сетевых интерфейсов на сервере осуществляется с использованием утилиты netplan. Конфигурационный файл /etc/netplan/00-installer-config.yaml определяет статический IP-адрес 10.11.9.1/24 для интерфейса внутренней сети enp0s8 и DHCP для интерфейса enp0s3,

обеспечивающего доступ к сети Интернет. Применение конфигурации выполняется командой sudo netplan apply. Итоговое содержимое конфигурационного файла приведено на Рисунке 4.6.

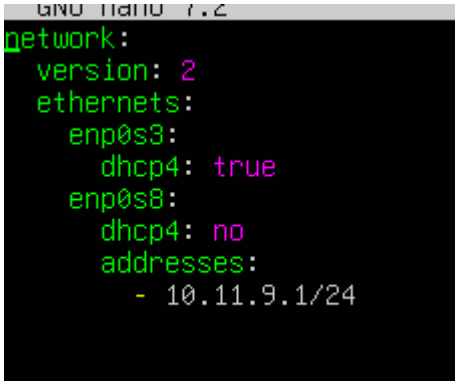


Рисунок 4.6 — Конфигурация сетевых интерфейсов сервера в файле /etc/netplan

Поскольку интерфейс enp0s3 сервера получает адрес по DHCP от NAT-адаптера VirtualBox, для обращения к внешним DNS-серверам (необходимо для обновления пакетов через Интернет) на сервере дополнительно задаётся резолвер в файле /etc/resolv.conf, как показано на Рисунке 4.7.

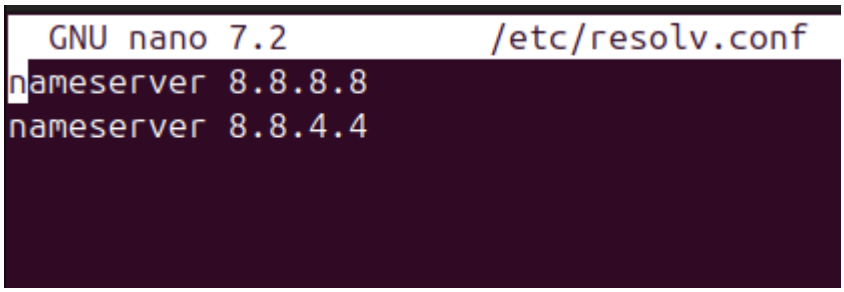


Рисунок 4.7 — Настройка DNS-резолвера сервера

Корректность применения сетевых настроек проверяется командой ip a, вывод которой подтверждает назначение адресов на обоих интерфейсах сервера (Рисунок 4.8).

```
dyakov@dyakov-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:2e:1d:bc brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft 86164sec preferred_lft 86164sec
    inet6 fd17:625c:f037:2:a00:27ff:fe2e:1dbc/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86236sec preferred_lft 14236sec
    inet6 fe80::a00:27ff:fe2e:1dbc/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:65:0e:94 brd ff:ff:ff:ff:ff:ff
    inet 10.11.9.1/24 brd 10.11.9.255 scope global enp0s8
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe65:e94/64 scope link
       valid_lft forever preferred_lft forever
dyakov@dyakov-server:~$
```

Рисунок 4.8 — Проверка сетевых настроек сервера

На АРМ тестировщика аналогичным образом настраивается сетевой интерфейс enp0s8 с адресом 10.11.9.2/24 для обеспечения связи с сервером по внутренней сети; конфигурация выполняется через NetworkManager и приведена на Рисунке 4.9.

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: no
      addresses:
        - 10.11.9.2/24
```

Рисунок 4.9 — Конфигурация сетевого интерфейса АРМ тестировщика

DNS-резолвер на АРМ настроен таким образом, чтобы в первую очередь использовался внутренний сервер 10.11.9.1, а в качестве резервных — публичные DNS-серверы (Рисунок 4.10).

```
nameserver 10.11.9.1
nameserver 8.8.8.8
nameserver 8.8.4.4
nameserver 127.0.0.53
options edns0 trust-ad
search .
```

Рисунок 4.10 — Настройка DNS-резолвера АРМ

Применение настроек на АРМ также подтверждается выводом команды ip a (Рисунок 4.11), после чего проверяется сетевая доступность сервера по внутренней сети с помощью команды ping — стабильное время отклика (1,75–4,70 мс) и отсутствие потерь пакетов подтверждают корректность настройки внутренней сети intnet (Рисунок 4.12).

```
dyakov@dyakov-arm:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen
1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group def
ault qlen 1000
    link/ether 08:00:27:18:3d:91 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86390sec preferred_lft 86390sec
    inet6 fe80::a00:27ff:fe18:3d91/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group def
ault qlen 1000
    link/ether 08:00:27:ad:d3:88 brd ff:ff:ff:ff:ff:ff
    inet 10.11.9.2/24 brd 10.11.9.255 scope global noprefixroute enp0s8
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fead:d388/64 scope link
       valid_lft forever preferred_lft forever
```

Рисунок 4.11 — Проверка сетевых настроек АРМ

```
dyakov@dyakov-arm:~$ ping 10.11.9.1
PING 10.11.9.1 (10.11.9.1) 56(84) bytes of data.
64 bytes from 10.11.9.1: icmp_seq=1 ttl=64 time=4.70 ms
64 bytes from 10.11.9.1: icmp_seq=2 ttl=64 time=2.17 ms
64 bytes from 10.11.9.1: icmp_seq=3 ttl=64 time=1.88 ms
64 bytes from 10.11.9.1: icmp_seq=4 ttl=64 time=1.75 ms
64 bytes from 10.11.9.1: icmp_seq=5 ttl=64 time=1.78 ms
^C
--- 10.11.9.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4040ms
rtt min/avg/max/mdev = 1.745/2.453/4.695/1.131 ms
```

Рисунок 4.12 — Пинг к серверу

Далее на сервере выполняется настройка служб DHCP и DNS средствами пакета dnsmasq. Перед редактированием конфигурации выполняется резервное копирование исходного файла /etc/dnsmasq.conf (Рисунок 4.13), после чего в конфигурационный файл вносятся параметры выдачи IP-адресов в диапазоне 10.11.9.10–10.11.9.50 и разрешения доменного имени openproject.local в адрес сервера 10.11.9.1 (Рисунок 4.14).

```
dyakov@dyakov-server:~$ sudo cp /etc/dnsmasq.conf /etc/dnsmasq.conf.backup
dyakov@dyakov-server:~$
```

Рисунок 4.13 — Создание резервной копии конфигурационного файла dnsmasq

```
# "/usr/sbin/dnsmasq --help" or "man 8 dnsmasq" for detai
interface=enp0s8
dhcp-range=10.11.9.10,10.11.9.50,12h
address=/openproject.local/10.11.9.1
# Listen on this specific port instead of the standard DN
# (53). Setting this to zero completely disables DNS func
```

Рисунок 4.14 — Настройка параметров DHCP-диапазона и DNS-разрешения в конфигурационном файле dnsmasq

После применения настроек служба dnsmasq запускается и добавляется в автозагрузку; вывод systemctl status подтверждает её активное состояние (Рисунок 4.15).

```
dyakov@dyakov-server:~$ sudo systemctl restart dnsmasq
dyakov@dyakov-server:~$ sudo systemctl enable dnsmasq
Synchronizing state of dnsmasq.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable dnsmasq
^[[Adyakov@dyakov-server
dyakov@dyakov-server:~$ sudo systemctl status dnsmasq
● dnsmasq.service - dnsmasq - A lightweight DHCP and caching DNS server
     Loaded: loaded (/usr/lib/systemd/system/dnsmasq.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-06-20 14:33:50 UTC; 17s ago
   Main PID: 4810 (dnsmasq)
      Tasks: 1 (limit: 4601)
     Memory: 760.0K (peak: 2.2M)
        CPU: 82ms
     CGroup: /system.slice/dnsmasq.service
             └─4810 /usr/sbin/dnsmasq -x /run/dnsmasq/dnsmasq.pid -u dnsmasq -r /run/dnsmasq/resolv.conf -7 /etc/dnsmasq.d,.dpkg-dist,.dpkg-old,.dpkg-new --loca
l-service --trust-anchor=.,20326,8,2,E06D44B80B8F1D39A95C0B0D7C65D08458E880409BBC683457104237C7F8EC8D --trust-anchor=.,38696,8,2,683D2D0ACB8C9B712A1948B27F74121
9298D0A450D612C483AF444A4C0FB2B16

Jun 20 14:33:50 dyakov-server systemd[1]: Starting dnsmasq.service - dnsmasq - A lightweight DHCP and caching DNS server...
Jun 20 14:33:50 dyakov-server dnsmasq[4810]: started, version 2.90 cachesize 150
Jun 20 14:33:50 dyakov-server dnsmasq[4810]: compile time options: IPv6 GNU-getopt DBus no-UBus i18n IDN2 DHCP DHCPv6 no-Lua TFTP conntrack ipset nftset auth cr
yptohash DNSSEC loop-detect inotify dumpfile
Jun 20 14:33:50 dyakov-server dnsmasq-dhcp[4810]: DHCP, IP range 10.11.9.10 -- 10.11.9.50, lease time 12h
Jun 20 14:33:50 dyakov-server dnsmasq[4810]: read /etc/hosts - 8 names
Jun 20 14:33:50 dyakov-server resolvconf[4818]: Dropped protocol specifier '.dnsmasq' from 'lo.dnsmasq'. Using 'lo' (ifindex=1).
Jun 20 14:33:50 dyakov-server resolvconf[4818]: Failed to set DNS configuration: Unit dbus-org.freedesktop.resolve1.service not found.
Jun 20 14:33:50 dyakov-server systemd[1]: Started dnsmasq.service - dnsmasq - A lightweight DHCP and caching DNS server.
```

Рисунок 4.15 — Подтверждение успешного запуска и активной работы службы DNS/DHCP сервера (dnsmasq)

Доступ по протоколу SSH настраивается на стандартном порту 22. В файле /etc/ssh/sshd_config запрещается вход для пользователя root и разрешается аутентификация пользователя dyakov по открытому ключу (Рисунок 4.16); парольная аутентификация на данном этапе временно сохранена для возможности первичного копирования ключа со стороны АРМ (см. п. 4.5) и подлежит отключению после переноса ключей на постоянной основе.

```
#   systemctl restart ssh.socket
#
Port 22
PermitRootLogin no
PasswordAuthentication yes
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
```

Рисунок 4.16 — Конфигурация файла sshd_config для обеспечения безопасного удаленного доступа

Для обеспечения сетевой безопасности на сервере применяется пакетный фильтр iptables с политикой «запрещено всё, что не разрешено явно»: по умолчанию все входящие соединения блокируются правилом iptables -P INPUT DROP, а доступ разрешён только для портов 22 (SSH), 80 и 443 (HTTP/HTTPS), 5432 (PostgreSQL), 53 (DNS), 67 (DHCP) и 19999 (Netdata). Настроенные правила сохраняются с помощью пакета iptables-persistent для автоматической загрузки при перезапуске системы (Рисунок 4.17).

```
dyakov@dyakov-server:~$ sudo iptables -F
dyakov@dyakov-server:~$ sudo iptables -X
dyakov@dyakov-server:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
dyakov@dyakov-server:~$ sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
dyakov@dyakov-server:~$ sudo iptables -A INPUT -i lo -j ACCEPT
dyakov@dyakov-server:~$ sudo iptables -P INPUT DROP
dyakov@dyakov-server:~$ sudo iptables -P FORWARD DROP
dyakov@dyakov-server:~$ sudo iptables -P OUTPUT ACCEPT
dyakov@dyakov-server:~$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
dyakov@dyakov-server:~$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
dyakov@dyakov-server:~$ sudo iptables -A INPUT -p tcp --dport 5432 -j ACCEPT
dyakov@dyakov-server:~$ sudo iptables -A INPUT -p udp --dport 53 -j ACCEPT
dyakov@dyakov-server:~$ sudo iptables -A INPUT -p udp --dport 67 -j ACCEPT
dyakov@dyakov-server:~$ sudo iptables -A INPUT -p tcp --dport 19999 -j ACCEPT
dyakov@dyakov-server:~$ sudo netfilter-persistent save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/15-ip4tables save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/25-ip6tables save
dyakov@dyakov-server:~$
```

Рисунок 4.17 — Настройка и сохранение правил межсетевого экрана (iptables) для обеспечения сетевой безопасности сервера

Для автоматизации последующих задач администрирования на АРМ тестировщика устанавливается безагентная система конфигурационного управления Ansible, работающая по протоколу SSH (Рисунок 4.18).

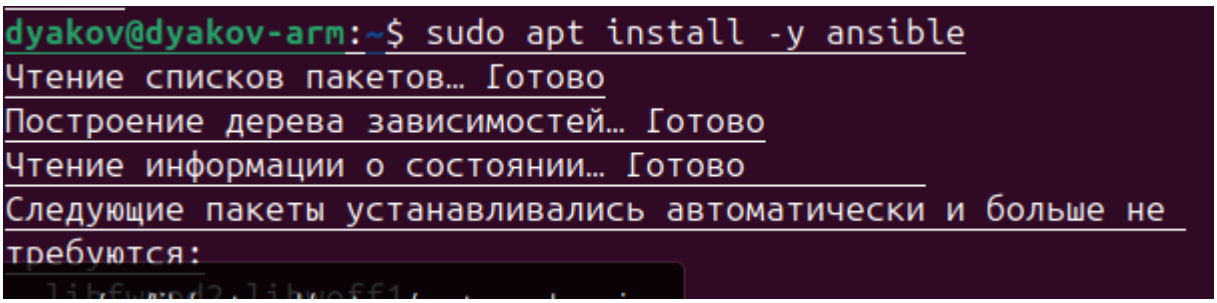


Рисунок 4.18 — Установка системы управления конфигурациями Ansible на АРМ тестировщика

Для работы Ansible на АРМ создаётся рабочая директория проекта и файл инвентаризации узлов inventory.ini (Рисунок 4.19), содержащий адрес управляемого сервера и параметры подключения (Рисунок 4.20).

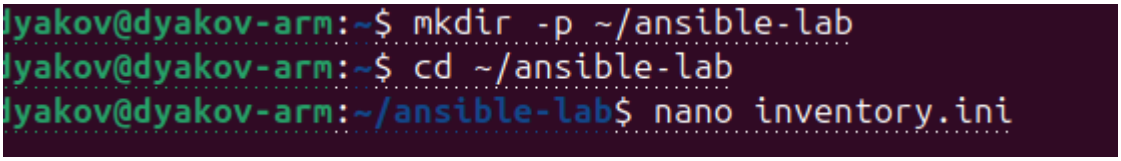


Рисунок 4.19 — Подготовка среды для работы с Ansible: создание директории проекта и файла описания узлов

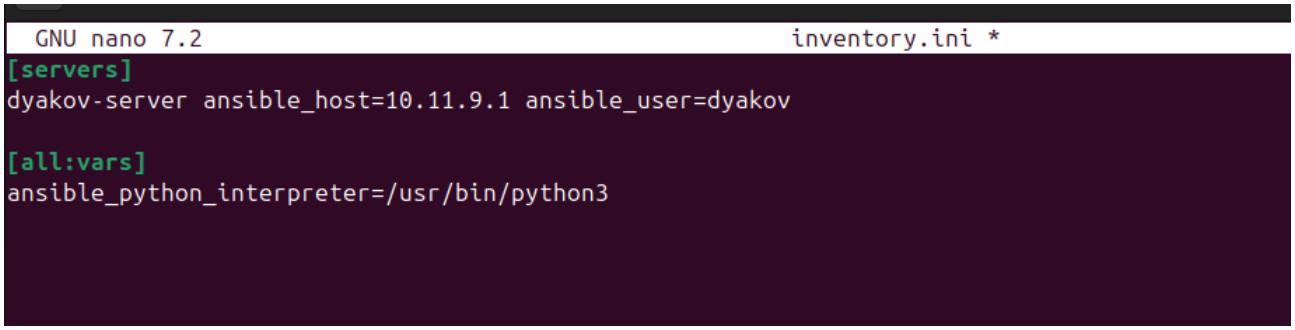


Рисунок 4.20 — Содержимое файла инвентаризации inventory.ini с описанием управляемого узла и параметрами подключения Ansible

Работоспособность связки Ansible — SSH проверяется модулем ping, который возвращает успешный ответ "pong" от сервера (Рисунок 4.21); далее создаётся сценарий автоматизации (playbook) setup-server.yml, выполняющий установку утилиты htop, создание тестового файла и сбор информации о системе (Рисунок 4.22).

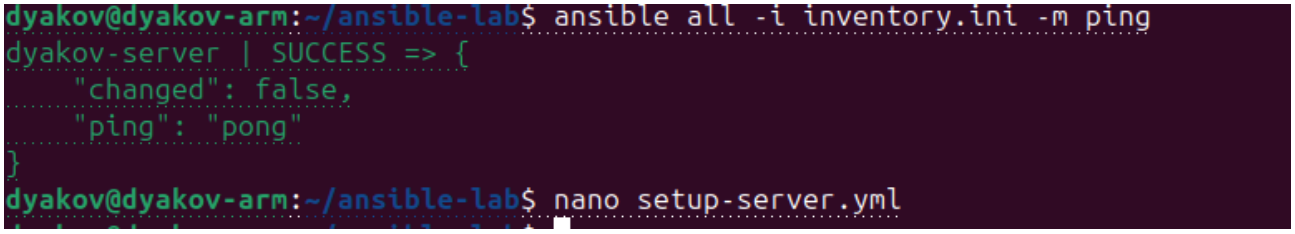


Рисунок 4.21 — Тестирование SSH-подключения и работоспособности агента Ansible на целевом сервере (результат выполнения модуля ping)

```yaml
---
- name: Настройка сервера тестирования
  hosts: servers
  become: yes

  tasks:
    - name: Установка htop
      apt:
        name: htop
        state: present
        update_cache: yes

    - name: Создание тестового файла
      copy:
        content: "Этот файл создан Ansible {{ ansible_date_time.iso8601 }}"
        dest: /tmp/ansible-test.txt
        owner: dyakov
        group: dyakov
        mode: '0644'

    - name: Вывод информации о системе
      command: uname -a
      register: system_info

    - name: Показ информации
      debug:
        var: system_info.stdout
```

Рисунок 4.22 — Содержимое сценария автоматизации setup-server.yml (Playbook Ansible)

Запуск playbook командой ansible-playbook выполняется без ошибок: все задачи (tasks) отрабатывают на сервере, что подтверждается итоговой статистикой PLAY RECAP без неудачных операций (Рисунок 4.23). Корректность результата дополнительно проверяется прямым подключением к серверу по SSH и просмотром содержимого созданного файла /tmp/ansible-test.txt (Рисунок 4.24).

```
dyakov@dyakov-arm:~/ansible-lab$ ansible-playbook -i inventory.ini setup-server.yml

PLAY [Настройка сервера тестирования] ************************************

TASK [Gathering Facts] ***************************************************
ok: [dyakov-server]

TASK [Установка htop] ****************************************************
ok: [dyakov-server]

TASK [Создание тестового файла] ******************************************
changed: [dyakov-server]

TASK [Вывод информации о системе] ****************************************
changed: [dyakov-server]

TASK [Показ информации] **************************************************
ok: [dyakov-server] => {
    "system_info.stdout": "Linux dyakov-server 6.8.0-124-generic #124-Ubuntu SMP PREEMPT_DYNAMIC Tue May 26 13:00:45 UTC
 2026 x86_64 x86_64 x86_64 GNU/Linux"
}

PLAY RECAP ***************************************************************
dyakov-server              : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

dyakov@dyakov-arm:~/ansible-lab$
```

Рисунок 4.23 — Результат выполнения сценария автоматизации Ansible (Playbook) на целевом сервере

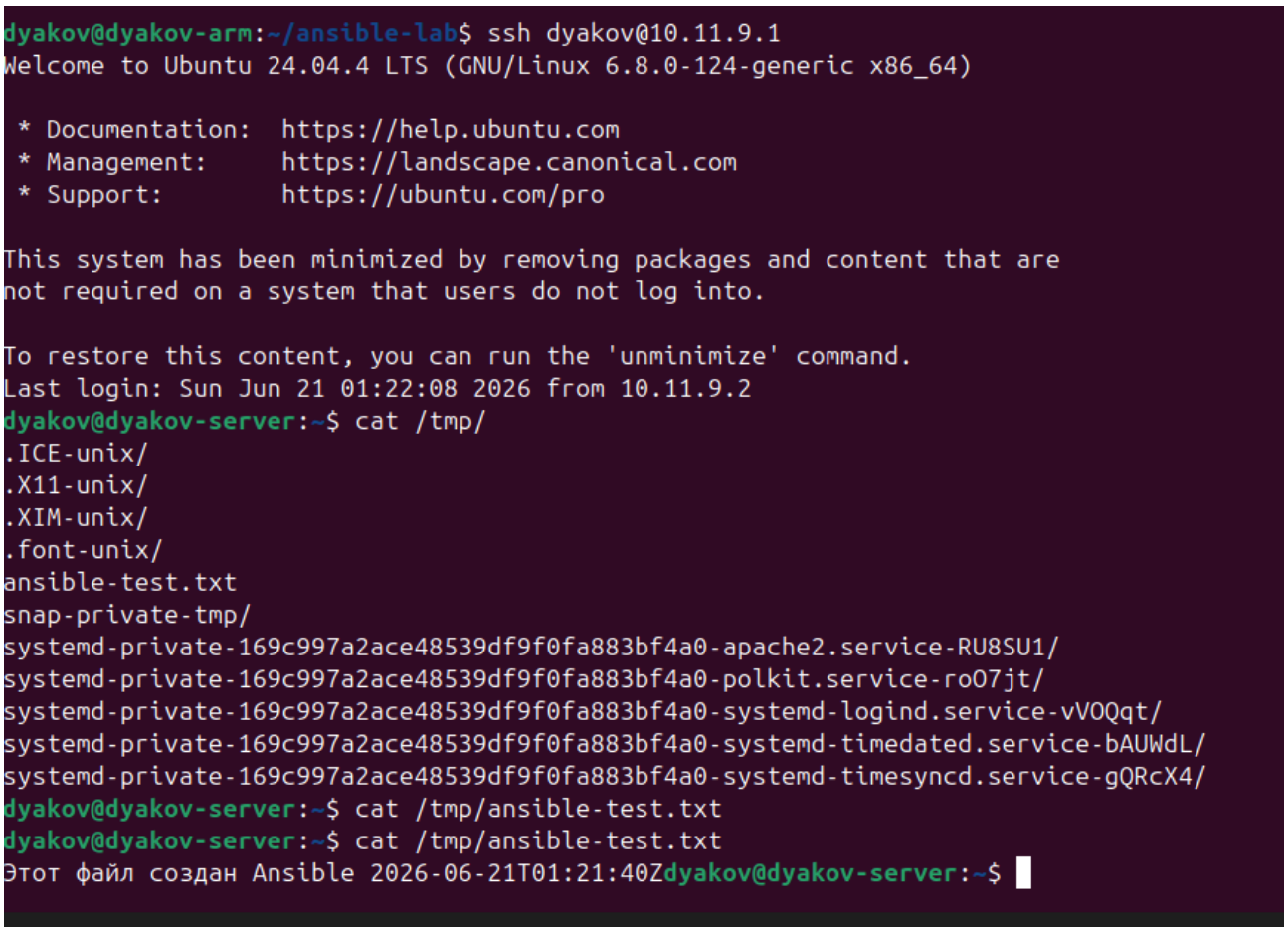


Рисунок 4.24 — Проверка результатов выполнения Ansible Playbook на целевом сервере

Завершающим шагом базовой настройки служб является общая проверка состояния сервера: утилита htop отображает текущую загрузку процессора, использование оперативной памяти и активные процессы, включая ранее запущенные службы (Рисунок 4.25).

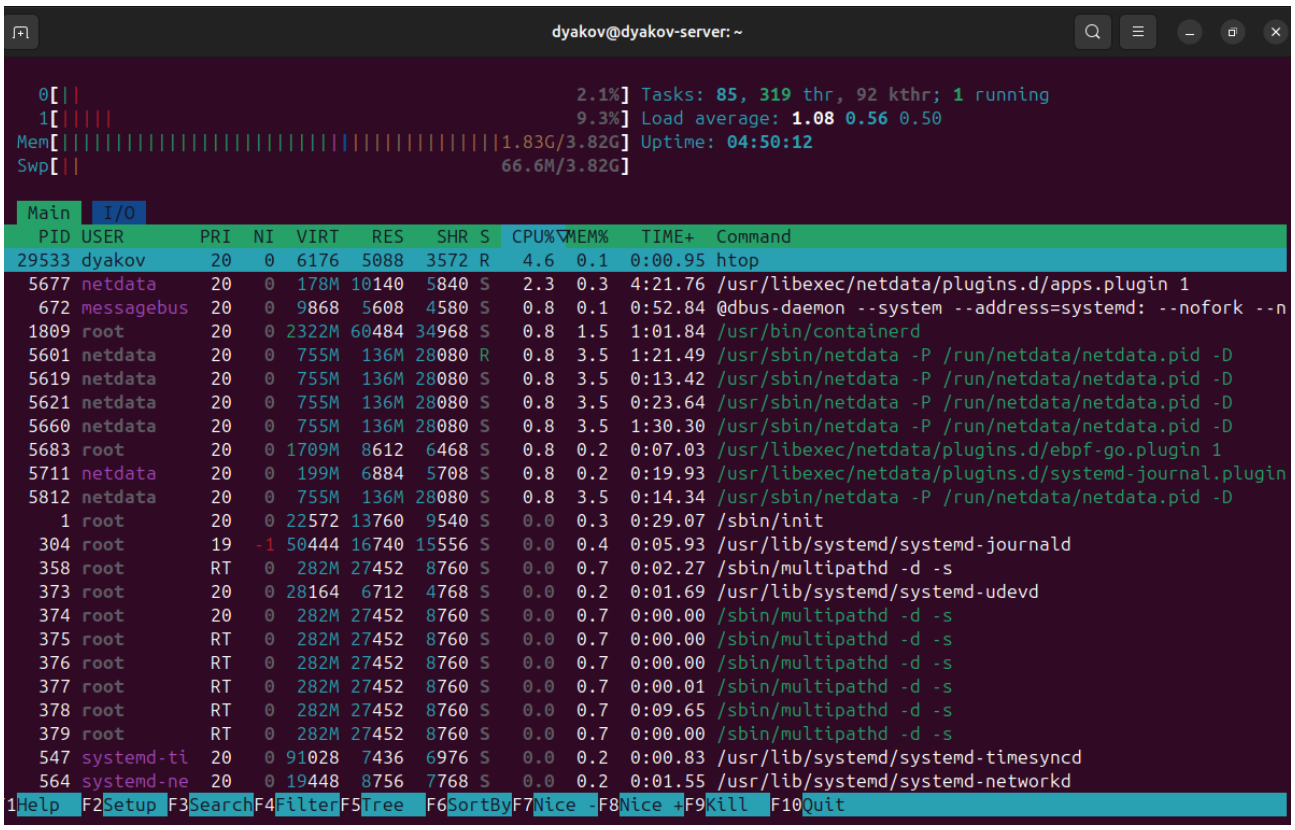


Рисунок 4.25 — Мониторинг системных ресурсов и активных процессов сервера с помощью утилиты htop

4.4 Развертывание серверного прикладного ПО

Для обеспечения контейнеризации приложений на сервере устанавливается платформа Docker. Перед добавлением официального репозитория устанавливаются базовые утилиты для безопасной загрузки пакетов — ca-certificates, curl и gnupg (Рисунок 4.26), после чего добавляется GPG-ключ и репозиторий Docker, и выполняется установка компонентов Docker CE, CLI, Containerd и Compose-плагина (Рисунок 4.27).

```
dyakov@dyakov-server:~$ sudo apt install -y ca-certificates curl gnupg
```

Рисунок 4.26 — Установка базовых утилит для безопасной загрузки пакетов и работы с репозиториями (ca-certificates, curl, gnupg)

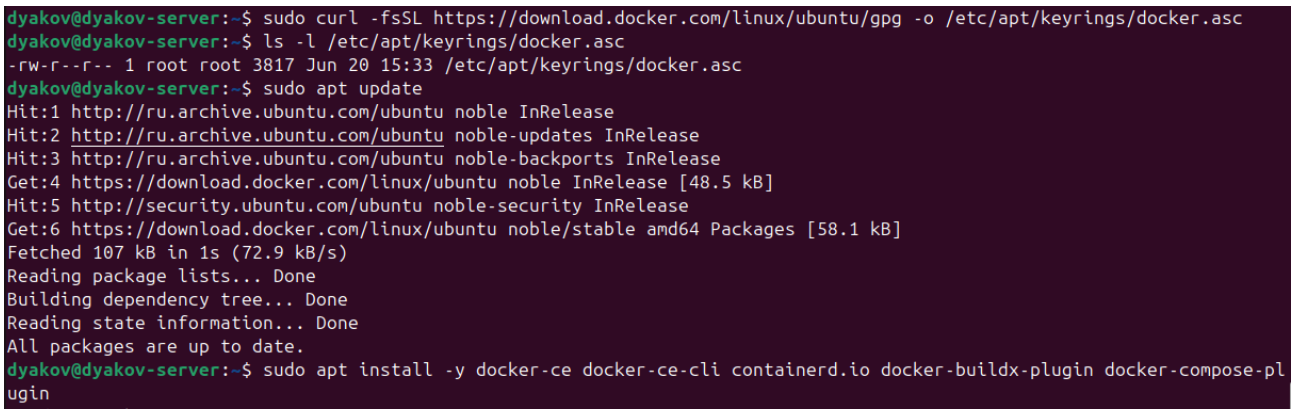


Рисунок 4.27 — Добавление официального репозитория Docker и установка компонентов контейнеризации (Docker CE, CLI, Containerd, Compose)

После установки служба Docker активируется и добавляется в автозагрузку, пользователь dyakov включается в группу docker для работы без повышения привилегий; версии Docker и Docker Compose подтверждаются соответствующими командами (Рисунок 4.28).

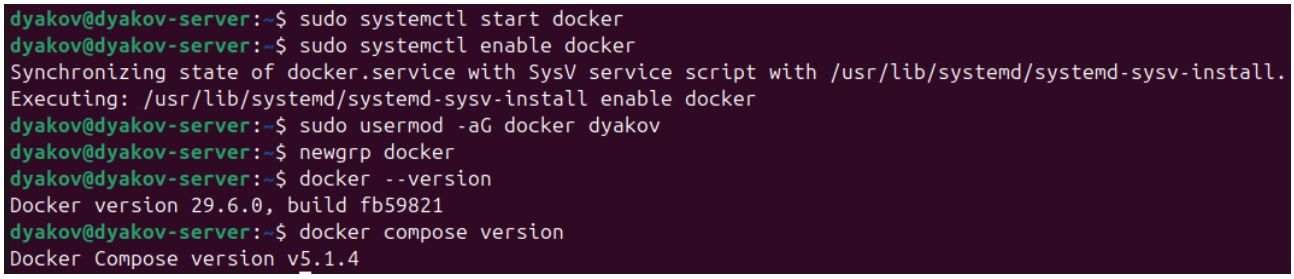


Рисунок 4.28 — Инициализация среды контейнеризации: активация демона Docker и верификация корректности установки

Система управления проектами OpenProject разворачивается с использованием Docker Compose в отдельной рабочей директории (Рисунок 4.29). Конфигурационный файл docker-compose.yml описывает два сервиса: контейнеризированную базу данных PostgreSQL (используется исключительно стеком OpenProject и не является той же СУБД, что устанавливается в п. 4.6 для учебной базы course_work) и веб-приложение OpenProject, связанные через переменные окружения и общую сеть Docker (Рисунок 4.30).

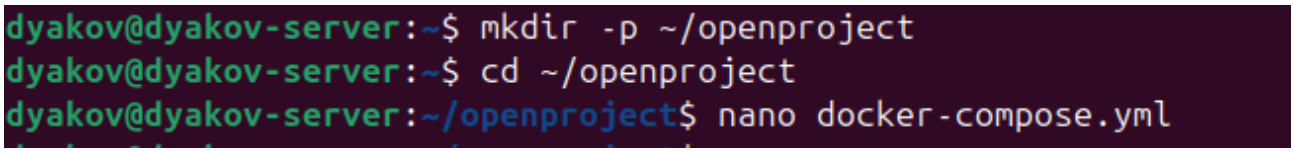


Рисунок 4.29 — Создание рабочей директории и подготовка файла конфигурации Docker Compose для развертывания OpenProject

```yaml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: openproject
      POSTGRES_USER: openproject
      POSTGRES_PASSWORD: openproject
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: always

  openproject:
    image: openproject/openproject:14
    ports:
      - "80:80"
    environment:
      DATABASE_URL: postgres://openproject:openproject@db:5432/openproject
      OPENPROJECT_HTTPS: "false"
      OPENPROJECT_HOST__NAME: 10.11.9.1
    volumes:
      - opdata:/var/openproject/assets
    depends_on:
      - db
    restart: always

volumes:
  pgdata:
  opdata:
```

Рисунок 4.30 — Конфигурация стека сервисов OpenProject в файле docker-compose.yml

Стек контейнеров запускается в фоновом режиме командой docker compose up -d (Рисунок 4.31). После запуска проверяется статус контейнеров командой docker compose ps — оба сервиса находятся в состоянии Up, а тестовый запрос curl к локальному порту 80 подтверждает, что веб-сервер OpenProject принимает соединения (код ответа 400 в данном случае ожидаем, так как curl выполнен без указания корректного заголовка Host, и свидетельствует именно о доступности сервиса, а не о его неработоспособности) (Рисунок 4.32).

```
dyakov@dyakov-server:~/openproject$ docker compose down
dyakov@dyakov-server:~/openproject$ docker compose up -d
[+] up 8/36
```

Рисунок 4.31 — Запуск стека контейнеров OpenProject в фоновом режиме с помощью Docker Compose

```
dyakov@dyakov-server:~/openproject$ docker compose ps
NAME                          IMAGE                          COMMAND                  SERVICE       CREATED         STATUS
       PORTS
openproject-db-1              postgres:15                    "docker-entrypoint.s…"   db            6 minutes ago   Up 6 min
utes   5432/tcp
openproject-openproject-1     openproject/openproject:14     "./docker/prod/entry…"   openproject   6 minutes ago   Up 6 min
utes   0.0.0.0:80->80/tcp, [::]:80->80/tcp
dyakov@dyakov-server:~/openproject$ curl -I http://localhost:80
HTTP/1.1 400 Bad Request
Date: Sat, 20 Jun 2026 16:09:31 GMT
Server: Apache/2.4.62 (Debian)
Content-Type: text/plain
Content-Length: 31
Connection: close
```

Рисунок 4.32 — Проверка статуса запущенных контейнеров и тестирование доступности веб-сервера OpenProject

Доступность OpenProject подтверждается из браузера на АРМ тестировщика: по адресу сервера открывается страница авторизации (Рисунок 4.33), а после входа под учётной записью администратора отображается главная панель управления (Dashboard) с разделами проектов, задач и канбан-досок (Рисунок 4.34).

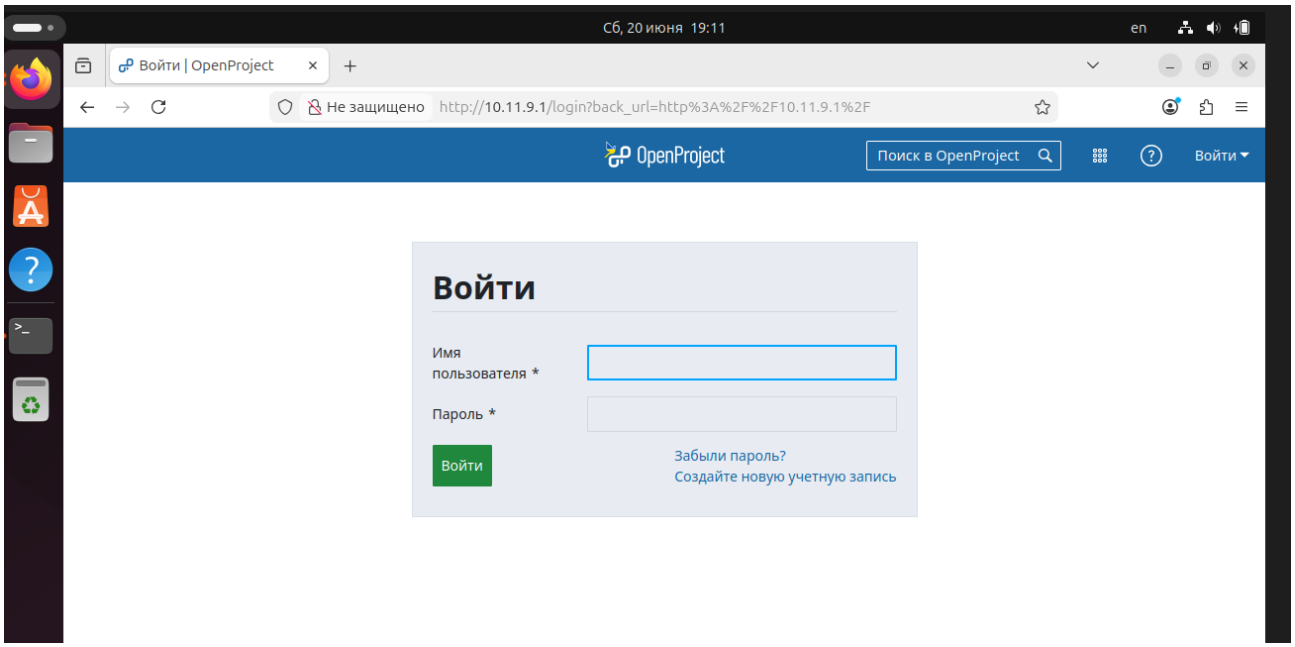


Рисунок 4.33 — Веб-интерфейс системы управления проектами OpenProject: страница авторизации пользователей

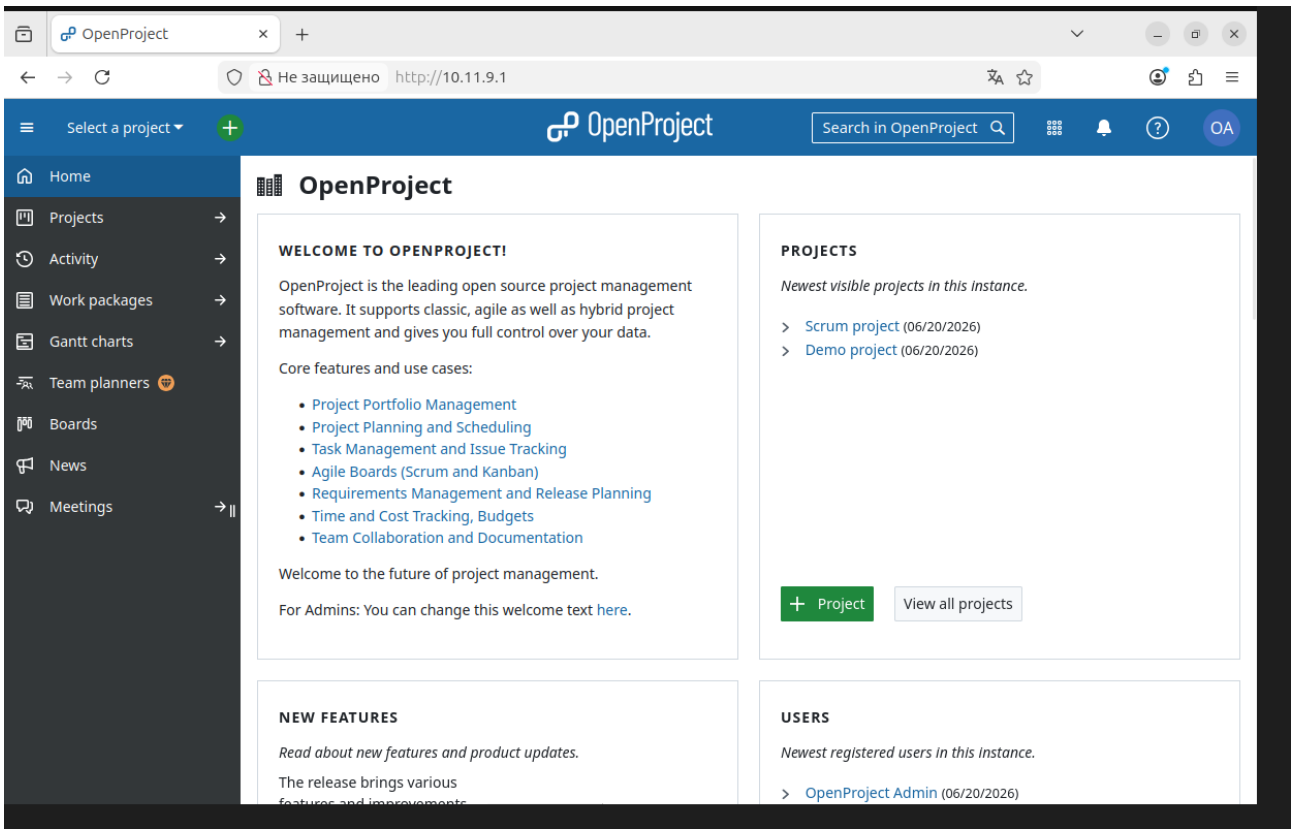


Рисунок 4.34 — Главная панель управления (Dashboard) системы OpenProject после успешной авторизации

Для мониторинга ресурсов сервера устанавливается система Netdata с помощью официального скрипта автоматической установки, запускаемого в неинтерактивном режиме (Рисунок 4.35). После завершения установки проверяется статус сопутствующей системной службы — netdata.service находится в активном состоянии (Рисунок 4.36).

```
dyakov@dyakov-server:~$ wget -O /tmp/netdata-kickstart.sh https://get.netdata.cloud/kickstart.sh && sh /tmp/netdata-kick
start.sh --non-interactive
--2026-06-20 16:15:31--  https://get.netdata.cloud/kickstart.sh
Resolving get.netdata.cloud (get.netdata.cloud)... 8.47.69.6, 8.6.112.6, 2a06:98c1:3122:8000::6, ...
```

Рисунок 4.35 — Загрузка и запуск скрипта автоматической установки системы мониторинга Netdata

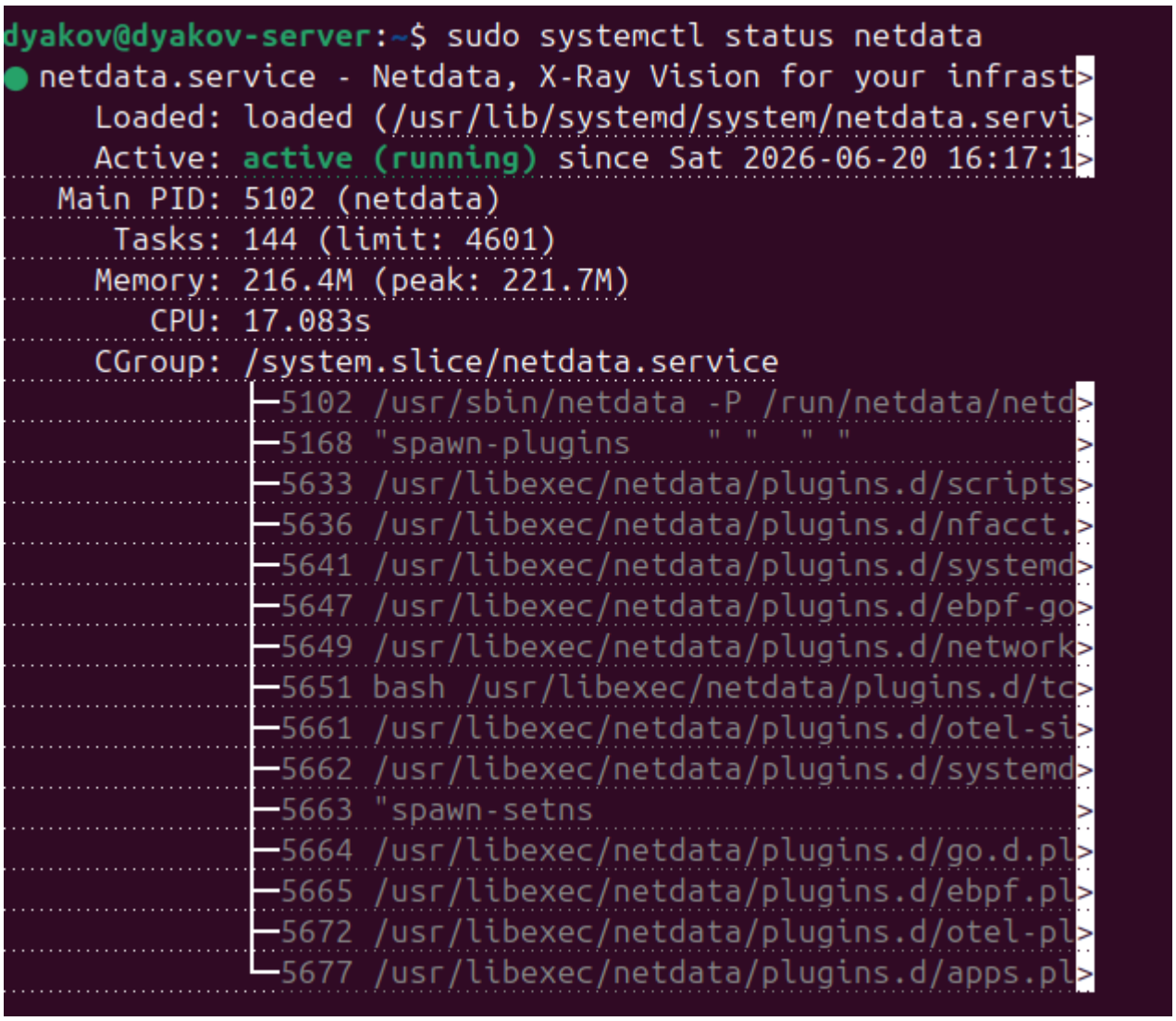


Рисунок 4.36 — Проверка статуса службы мониторинга Netdata (systemctl status netdata)

Веб-интерфейс Netdata, доступный по порту 19999, предоставляет обзор собираемых метрик и параметров хранения исторических данных в реальном времени (Рисунок 4.37).

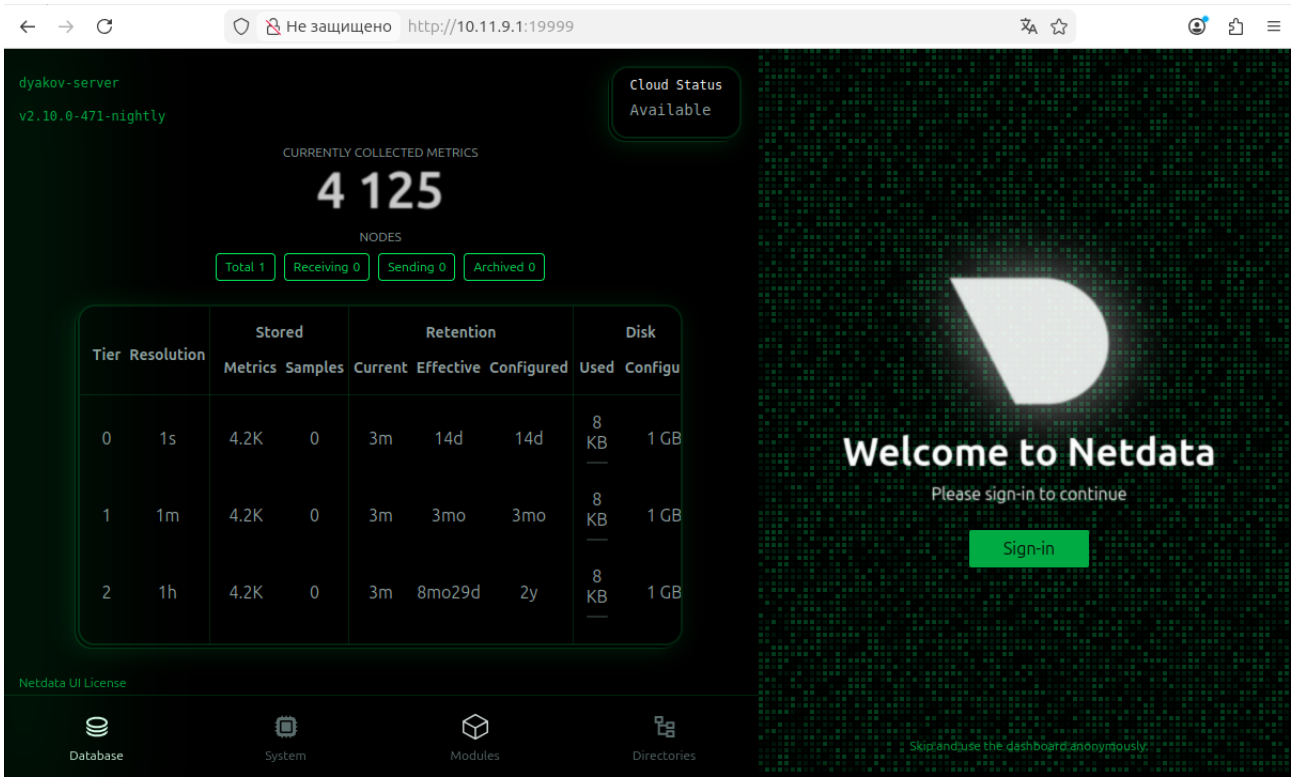


Рисунок 4.37 — Веб-интерфейс системы мониторинга Netdata: главная панель с обзором собираемых метрик и параметров хранения данных

4.5 Развертывание прикладного ПО на АРМ тестировщика

На АРМ тестировщика устанавливается набор инструментов для выполнения задач QA-инженера. Веб-браузер Firefox используется для доступа к веб-интерфейсу OpenProject и Netdata. Для тестирования API применяется Hoppscotch, устанавливаемый через DEB-пакет с официального сайта разработчика. Для работы с базой данных PostgreSQL используется графический клиент DBeaver Community Edition, также устанавливаемый через DEB-пакет. Дополнительно устанавливаются Python 3 и Git для выполнения скриптов автоматизации и работы с репозиториями. Последовательность команд установки перечисленных инструментов приведена в Листинге 4.1.

Листинг 4.1 — Установка базового программного обеспечения и инструментов QA на автоматизированное рабочее место (АРМ)

```
sudo apt update && sudo apt upgrade -y

sudo apt install -y python3 python3-pip git

cd ~/Загрузки
sudo apt install ./Hoppscotch_linux_x64.deb

wget -O /tmp/dbeaver.deb https://dbeaver.io/files/dbeaver-ce_latest_amd64.deb
sudo apt install -y /tmp/dbeaver.deb
```

Корректность установки инструмента API-тестирования подтверждается запуском Hoppscotch и подготовкой тестового HTTP-запроса к демонстрационному эндпоинту (Рисунок 4.38).

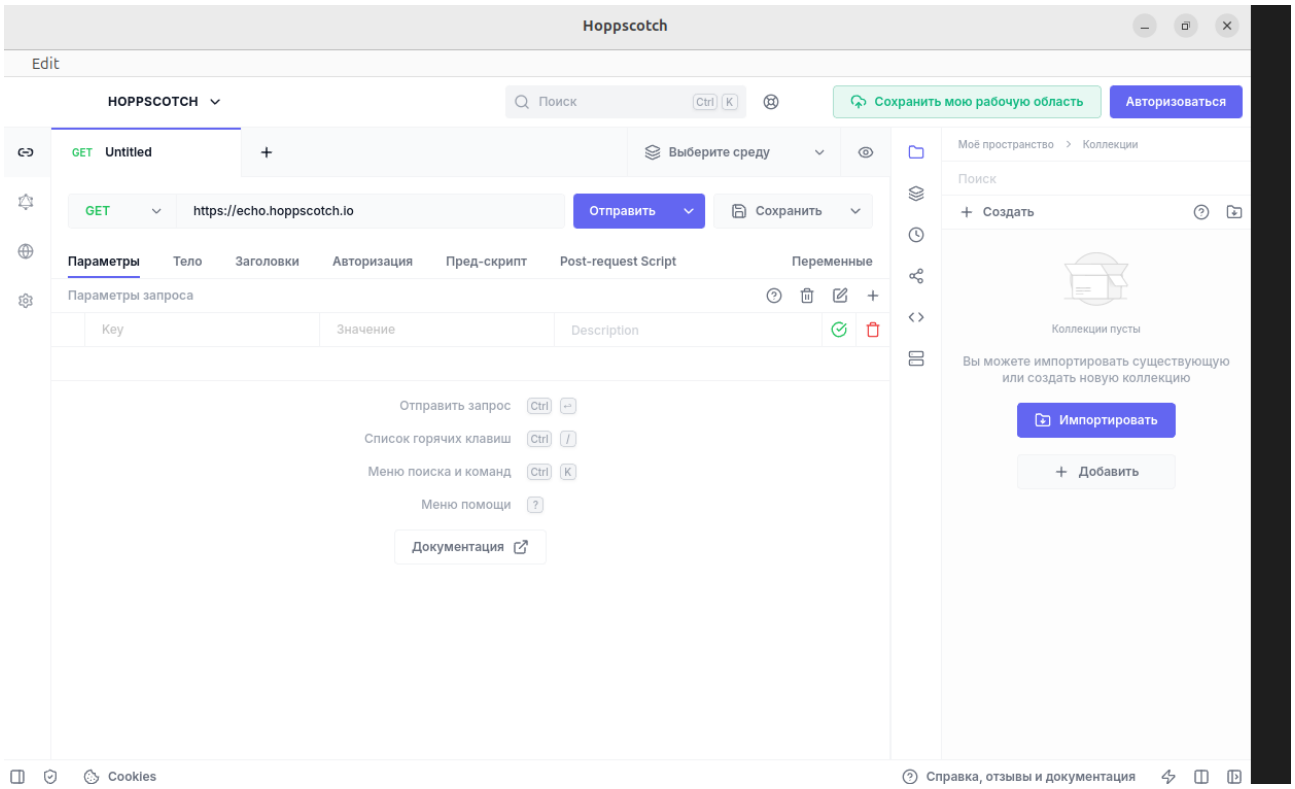


Рисунок 4.38 — Интерфейс инструмента тестирования API Hoppscotch: подготовка HTTP-запроса к тестовому эндпоинту

Для удаленного администрирования сервера на АРМ настраивается SSH-клиент: генерируется пара ключей RSA длиной 4096 бит, после чего публичный ключ копируется на сервер с помощью утилиты ssh-copy-id (Рисунок 4.39). После переноса ключа подключение к серверу выполняется без запроса пароля, что подтверждает работоспособность ключевой аутентификации (Рисунок 4.40).

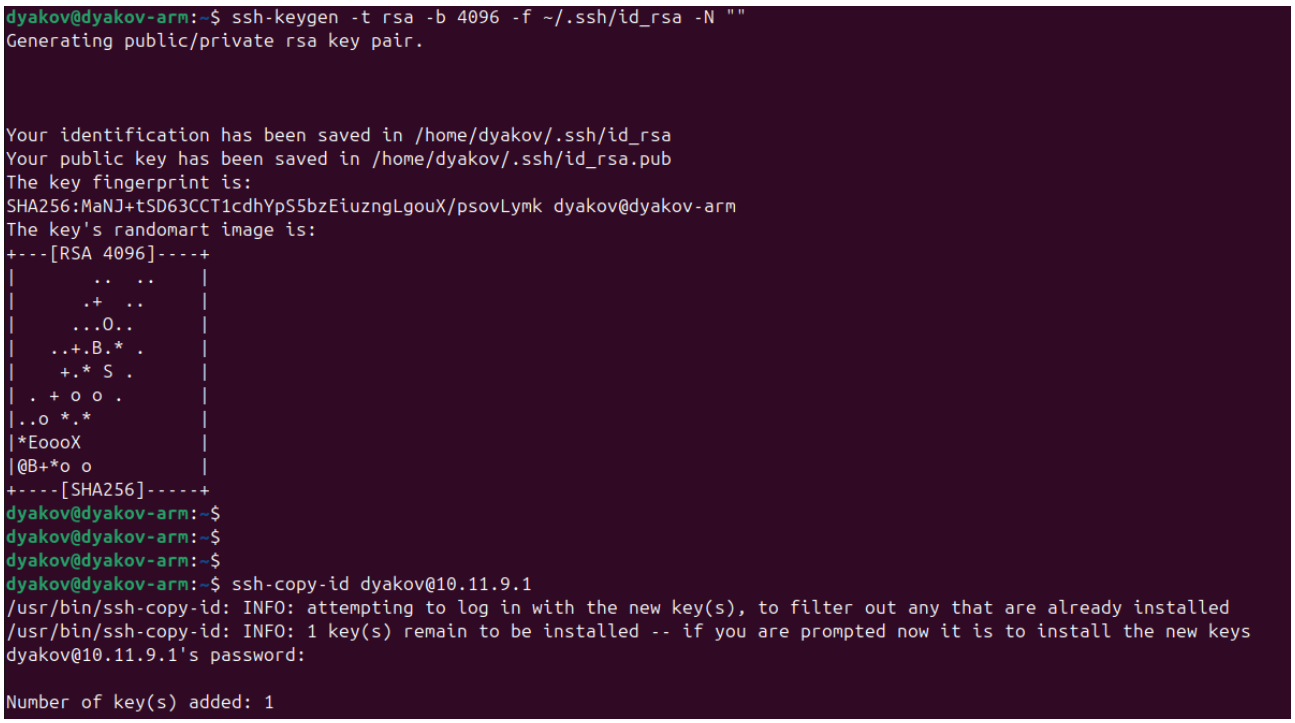


Рисунок 4.39 — Генерация пары ключей RSA и настройка беспарольной SSH-аутентификации с АРМ на сервер тестирования



Рисунок 4.40 — Успешное подключение к серверу тестирования по протоколу SSH с использованием ключевой аутентификации (без ввода пароля)

4.6 Развертывание системы управления базами данных

Для хранения и управления структурированными данными на сервере тестирования устанавливается отдельный (не связанный с контейнером OpenProject, см. п. 4.4) экземпляр системы управления базами данных PostgreSQL. Установка производится из официального репозитория Ubuntu с использованием пакетного менеджера APT. PostgreSQL обеспечивает надежное

хранение данных, поддержку транзакций и удаленный доступ через сетевой интерфейс.

Для демонстрации возможностей удаленного администрирования создаются тестовый пользователь student, база данных course_work и таблица students, наполняемая демонстрационными записями (Рисунок 4.41).

```
dyakov@dyakov-server:~$ sudo -u postgres psql -c "CREATE USER student WITH PASSWORD 'student123';"
CREATE ROLE
dyakov@dyakov-server:~$ sudo -u postgres psql -c "CREATE DATABASE course_work OWNER student;"
CREATE DATABASE
dyakov@dyakov-server:~$ sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE course_work TO student;"
GRANT
dyakov@dyakov-server:~$ sudo -u postgres psql -d course_work -c "
> REATE TABLE studen^C (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    group_name VARCHAR(50),
    grade INTEGER
);

INSERT INTO students (name, group_name, grade) VALUES
('Иванов Иван', 'ИС-21', 4),
('Петров Петр', 'ИС-21', 5),
('Сидоров Сидор', 'ИС-22', 3)
```

Рисунок 4.41 — Инициализация тестовой базы данных PostgreSQL: создание пользователя, схемы и наполнение таблицы данными

Для предоставления доступа из внутренней сети в конфигурационном файле postgresql.conf включается сетевое прослушивание на всех интерфейсах (Рисунок 4.42), а в файле pg_hba.conf добавляется правило, разрешающее подключения из подсети 10.11.9.0/24 с аутентификацией по паролю (md5) (Рисунок 4.43).

```
# - Connection Settings -

listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of add
                                # defaults to 'localhost'; us
                                # (change requires restart)
port = 5432                     # (change requires restart)
max_connections = 100           # (change requires restart)
#reserved_connections = 0       # (change requires restart)
```

Рисунок 4.42 — Настройка сетевого прослушивания в конфигурационном файле postgresql.conf

```
local   replication     all                                     peer
host    replication     all             127.0.0.1/32            scram-sha-256
host    replication     all             ::1/128                 scram-sha-256
host    all             all             10.11.9.0/24            md5
```

Рисунок 4.43 — Настройка правил аутентификации клиентов в файле pg_hba.conf

Для применения внесённых изменений служба PostgreSQL перезапускается (Рисунок 4.44). Поскольку порт 5432 уже был разрешён в правилах iptables на этапе базовой настройки (см. Рисунок 4.17), дополнительные правила межсетевого экрана не требуются; ниже приведено контрольное сохранение текущей конфигурации iptables, выполненное после изменений в PostgreSQL, чтобы исключить случайную потерю правил при последующей перезагрузке сервера (Рисунок 4.45).

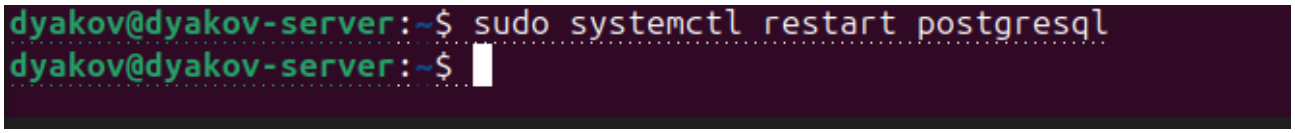


Рисунок 4.44 — Перезапуск службы PostgreSQL для применения изменений конфигурации

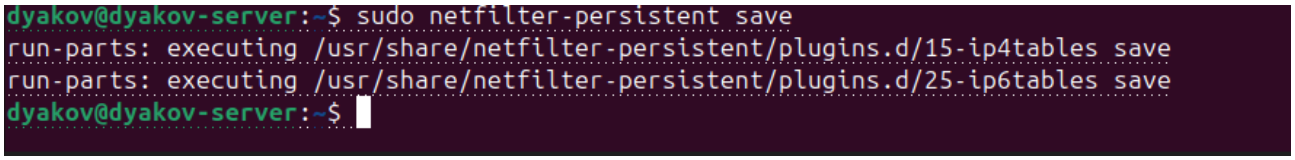


Рисунок 4.45 — Контрольное сохранение правил межсетевого экрана iptables после изменения конфигурации PostgreSQL

На АРМ тестировщика для работы с базой данных используется графический клиент DBeaver Community Edition, обеспечивающий визуальное управление базами данных, выполнение SQL-запросов, просмотр и редактирование данных в таблицах. Подключение к серверу PostgreSQL осуществляется по протоколу TCP через порт 5432 с использованием учетных данных пользователя student; тестовое подключение завершается успешно, что подтверждается информационным окном с версией сервера и драйвера (Рисунок 4.46).

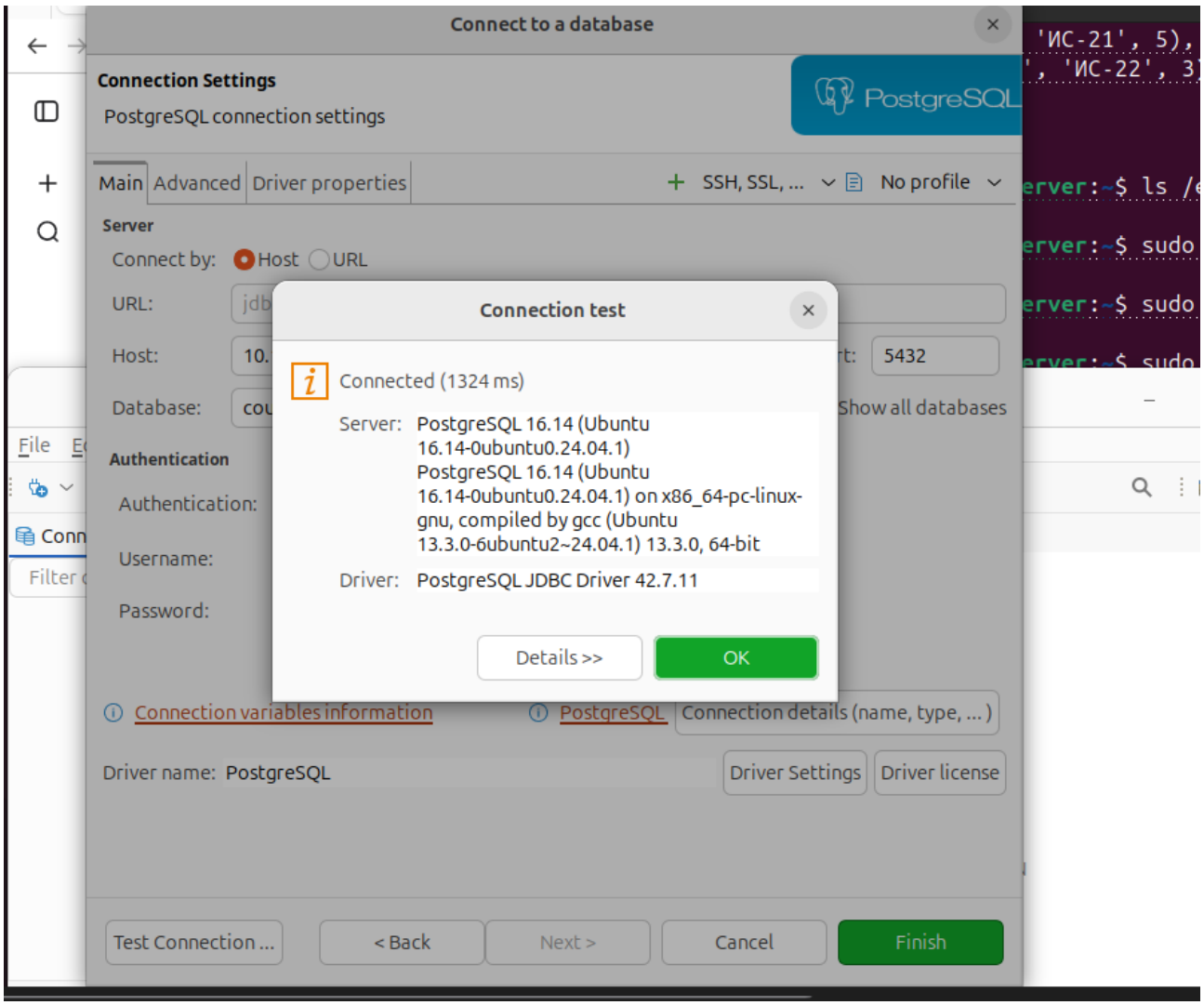


Рисунок 4.46 — Успешное тестовое подключение к удаленной базе данных PostgreSQL через графический клиент DBeaver

Завершающим шагом является визуальная проверка структуры развернутой базы данных: в навигаторе DBeaver отображается дерево объектов схемы course_work, а на диаграмме таблицы students показаны её поля и типы данных, что подтверждает корректность созданной структуры (Рисунок 4.47).

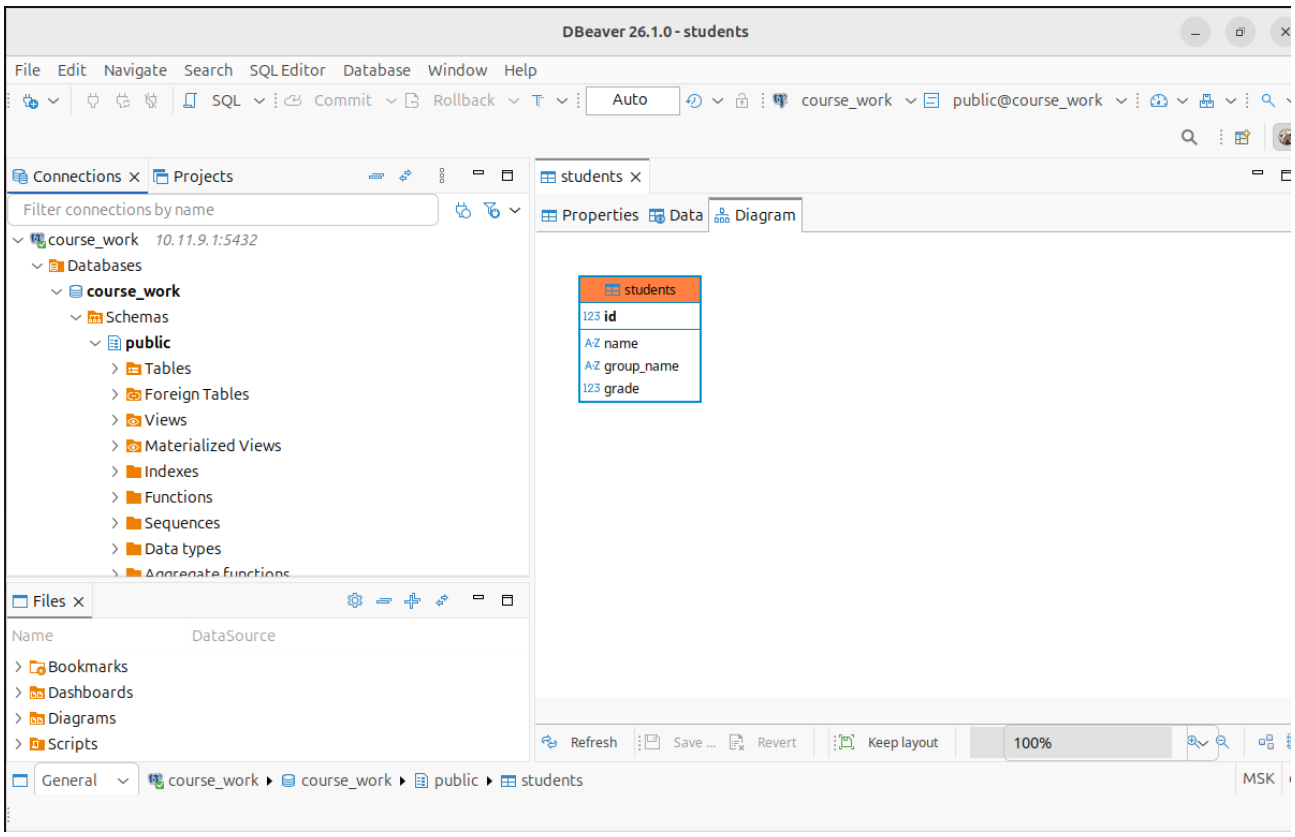


Рисунок 4.47 — Визуализация схемы БД course_work в DBeaver: навигация по объектам и диаграмма таблицы students

4.7 Тестирование функционального прототипа инфраструктуры

После завершения развертывания всех компонентов инфраструктуры выполняется комплексное тестирование функционального прототипа. Цель тестирования — подтверждение корректности работы всех сервисов, проверка их взаимодействия и соответствия заданным требованиям.

4.7.1 Функциональное тестирование серверных компонентов

Функциональное тестирование серверных компонентов включает проверку доступности и работоспособности всех развернутых служб. Для каждого сервиса выполняется проверка отклика на стандартные запросы и анализ логов на наличие ошибок.

Проверка службы DNS/DHCP (dnsmasq) осуществляется путем запроса разрешения доменного имени openproject.local с АРМ тестировщика. Команда

nslookup openproject.local 10.11.9.1 должна вернуть адрес сервера 10.11.9.1, что подтверждает корректную работу DNS-сервера (Рисунок 4.48).

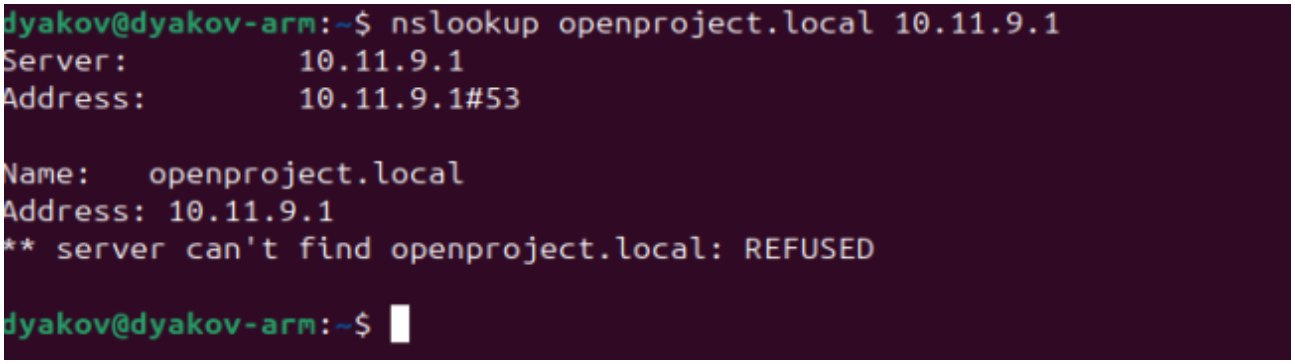


Рисунок 4.48 — Проверка работы DNS-сервера dnsmasq с использованием утилиты nslookup

Тестирование веб-сервиса OpenProject выполняется через браузер Firefox на АРМ. Проверяется доступность страницы авторизации по адресу http://10.11.9.1, корректность отображения элементов интерфейса и возможность успешной аутентификации под учетной записью администратора. После входа в систему выполняется навигация по основным разделам: проекты, задачи, канбан-доски (Рисунок 4.49).

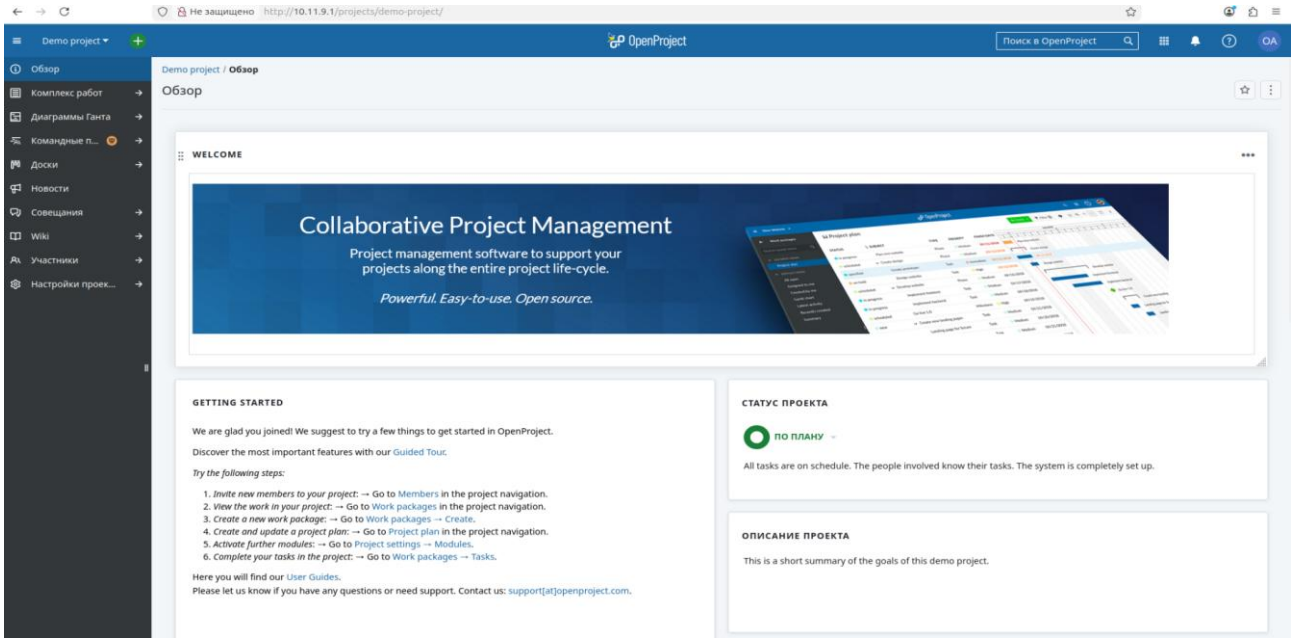


Рисунок 4.49 — Функциональное тестирование веб-интерфейса OpenProject: проверка навигации по разделам системы

Проверка системы мониторинга Netdata осуществляется путем обращения к веб-интерфейсу по адресу http://10.11.9.1:19999. Визуально проверяется наличие графиков в реальном времени, отображение метрик использования

процессора, оперативной памяти, дисковой подсистемы и сетевых интерфейсов. Отсутствие ошибок в консоли браузера и корректное обновление графиков подтверждают работоспособность службы мониторинга (Рисунок 4.50).



Рисунок 4.50 — Проверка системы мониторинга Netdata: визуализация метрик ресурсов сервера в реальном времени

4.7.2 Тестирование API тестируемого сервиса

Для проверки функциональности API системы бронирования (эмулируемой в рамках данного прототипа) используется инструмент Hoppscotch. Формируются тестовые HTTP-запросы к демонстрационному эндпоинту https://echo.hoppscotch.io для проверки корректности отправки GET, POST, PUT и DELETE запросов.

Тестовый GET-запрос с параметрами ?test=booking&user=student возвращает ожидаемый ответ с кодом состояния 200 OK и телом ответа, содержащим переданные параметры. Это подтверждает работоспособность инструмента API-тестирования и корректность сетевой настройки APM (Рисунок 4.51).

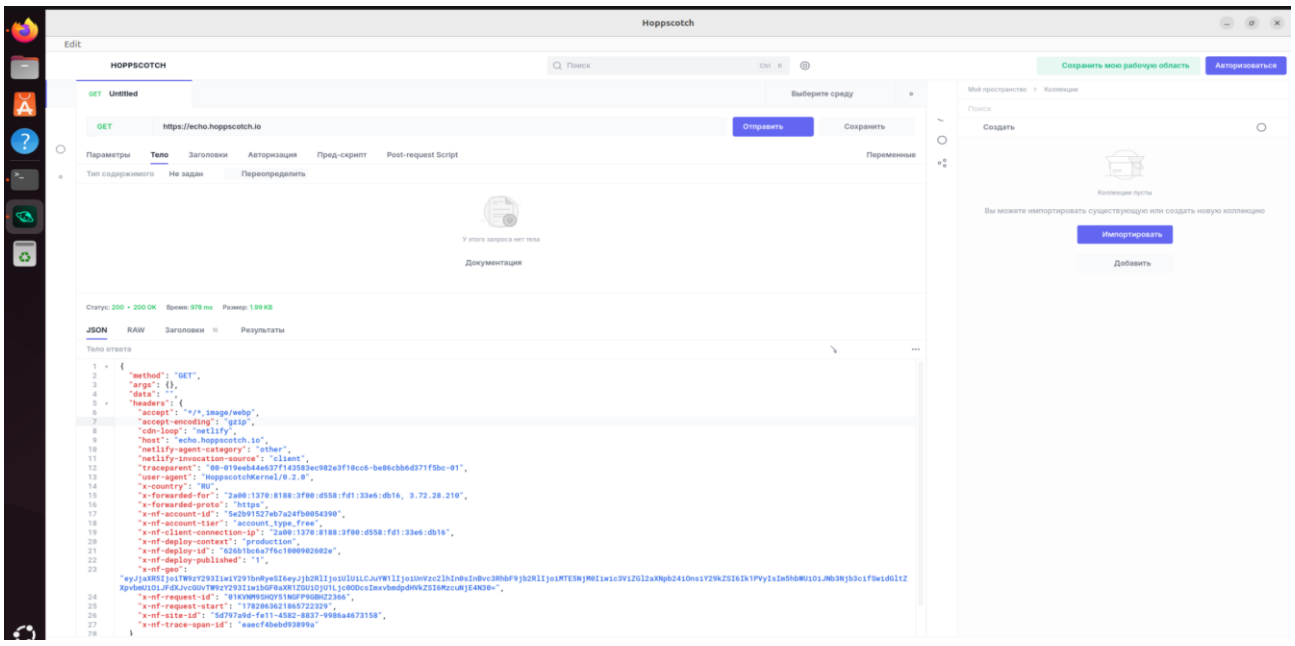


Рисунок 4.51 — Тестирование API через Hoppscotch: отправка GET-запроса и проверка ответа сервера

4.7.3 Тестирование производительности и нагрузки

Для оценки производительности инфраструктуры выполняется мониторинг использования ресурсов сервера при одновременной работе нескольких сервисов. Утилита htop отображает загрузку процессора, потребление оперативной памяти и количество активных процессов в реальном времени.

При одновременной работе OpenProject, PostgreSQL, Netdata и dnsmasq общая загрузка процессора составляет приблизительно 50%, что находится в пределах допустимых значений (Рисунок 4.52).

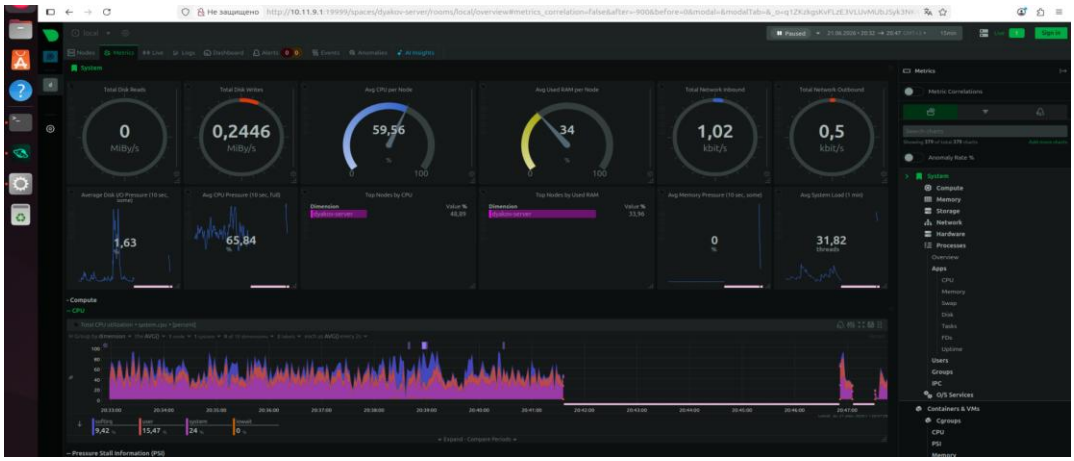


Рисунок 4.52 — Мониторинг производительности сервера: использование ресурсов при работе всех сервисов

4.7.4 Результаты тестирования

Результаты функционального тестирования всех компонентов инфраструктуры систематизированы в таблице 4.1. Для каждого компонента проверялась доступность, время отклика и корректность выполнения основных операций.

Таблица 4.1 — Результаты функционального тестирования компонентов инфраструктуры

Компонент	Проверяемый параметр	Ожидаемый результат	Фактический результат	Статус
dnsmasq (DNS)	Разрешение openproject.local	Возврат IP 10.11.9.1	IP 10.11.9.1 получен	Пройден
dnsmasq (DHCP)	Выдача IP-адресов	Диапазон 10.11.9.10-50	Адреса выдаются корректно	Пройден
OpenProject	Доступность веб-интерфейса	HTTP 200, страница входа	Страница загружена	Пройден
OpenProject	Аутентификация	Успешный вход	Вход выполнен	Пройден
PostgreSQL	Подключение к БД	Установлено соединение	Соединение активно	Пройден
PostgreSQL	Выполнение SQL-запроса	Возврат данных	Данные получены	Пройден
Netdata	Доступность мониторинга	Порт 19999 открыт	Веб-интерфейс доступен	Пройден
Docker	Статус контейнеров	Все контейнеры Up	2 контейнера активны	Пройден
SSH	Беспарольный вход	Вход без пароля	Аутентификация по ключу	Пройден
iptables	Фильтрация трафика	Закрыты лишние порты	Только разрешенные порты открыты	Пройден

4.7.5 Итоговая оценка функционального прототипа

По результатам комплексного тестирования функциональный прототип инфраструктуры для QA-тестировщика признается работоспособным и готовым к эксплуатации. Все компоненты инфраструктуры развернуты корректно, взаимодействие между сервисами настроено правильно, требования к производительности и безопасности соблюдены.

Выявленные в ходе тестирования характеристики позволяют сделать вывод о достаточности выделенных аппаратных ресурсов (1 vCPU, 8 ГБ RAM

для сервера) для функционирования учебного стенда. Использование контейнеризации Docker обеспечило изоляцию сервисов и упростило процесс развертывания. Применение системы управления конфигурациями Ansible позволило автоматизировать первоначальную настройку сервера и сократить время на развертывание инфраструктуры.

Развернутая инфраструктура полностью покрывает потребности QA-тестировщика в инструментах для выполнения функционального тестирования, регистрации дефектов, анализа логов и формирования отчетности, что подтверждает достижение поставленной цели курсовой работы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было выполнено комплексное проектирование и развертывание функционального прототипа инфраструктуры для QA-тестировщика на базе операционных систем семейства Linux. Рассмотрим основные результаты по каждому этапу работы.

В первом разделе был проведен анализ бизнес-процесса тестирования системы бронирования авиабилетов и отелей. Определены функциональные обязанности QA-инженера, охватывающие полный жизненный цикл проверки программного обеспечения: от анализа требований до формирования итоговой отчетности. Выполнена трехуровневая декомпозиция бизнес-процесса, позволившая детализировать конкретные операции, выполняемые тестировщиком на автоматизированном рабочем месте. Сформулированы нефункциональные требования к инфраструктуре, включающие показатели производительности (время отклика UI не более 2–3 секунд), надежности, удобства использования и информационной безопасности. Разработаны диаграммы взаимодействия компонентов и потоков данных, визуализирующие архитектурные решения.

Во втором разделе выполнено разделение программного обеспечения на прикладное и инструментальное классы. Проведен сравнительный анализ существующих на рынке решений для каждой категории задач: систем отслеживания дефектов (Jira, OpenProject, Redmine, Kaiten), инструментов API-тестирования (Postman, Hoppscotch, curl), веб-браузеров (Firefox, Chrome, Yandex Browser), клиентов СУБД (DataGrip, DBeaver, pgAdmin 4). Обоснован выбор свободного программного обеспечения: OpenProject в качестве системы управления проектами, Hoppscotch для API-тестирования, Mozilla Firefox для UI-тестирования, DBeaver Community Edition для работы с базами данных. Выбраны дистрибутивы Ubuntu Desktop 24.04 LTS для АРМ тестировщика и Ubuntu Server 24.04 LTS для серверной части, что обеспечивает стабильность, безопасность и совместимость со всем необходимым стеком технологий.

В третьем разделе спроектирована аппаратная инфраструктура на базе среды виртуализации Oracle VM VirtualBox. Для АРМ тестировщика выделены 1 vCPU, 4 ГБ оперативной памяти и 20 ГБ дискового пространства; для сервера — 1 vCPU, 8 ГБ оперативной памяти и 40 ГБ дискового пространства. Обоснована совместимость выбранной аппаратной платформы с ядром Linux, что исключает необходимость использования проприетарных драйверов. Спроектирована сетевая топология по принципу dual-homed архитектуры с использованием внутренней изолированной сети VirtualBox (intnet) для взаимодействия между АРМ и сервером. Разработана схема развертывания программного обеспечения и настроен пакетный фильтр iptables с политикой «запрещено всё, что не разрешено явно».

В четвертом разделе выполнено практическое развертывание функционального прототипа. Установлены и настроены операционные системы на обеих виртуальных машинах, сконфигурированы сетевые интерфейсы через netplan, развернуты службы DHCP и DNS на базе dnsmasq. На сервере установлены и запущены в контейнерах Docker система управления проектами OpenProject и СУБД PostgreSQL (отдельный экземпляр для учебной базы course_work). Развернута система мониторинга Netdata, предоставляющая графики ресурсов сервера в реальном времени. На АРМ тестировщика установлены инструменты QA-инженера: Firefox, Hoppscotch, DBeaver Community Edition, Python 3, Git. Настроена система автоматизации Ansible для удаленного управления сервером по протоколу SSH с аутентификацией по открытым ключам RSA. Проведено комплексное тестирование развернутой инфраструктуры, подтвердившее работоспособность всех компонентов: DNS-разрешение доменных имен, доступность веб-интерфейсов OpenProject и Netdata, возможность выполнения SQL-запросов через DBeaver, корректность работы API-тестирования в Hoppscotch.

Практическая значимость работы заключается в создании готового к использованию решения, которое может быть внедрено в образовательный процесс для подготовки QA-специалистов, а также адаптировано для нужд

небольших команд разработки, не располагающих бюджетом на коммерческое программное обеспечение. Все использованные технологии (Docker, Ansible, OpenProject, PostgreSQL, Netdata) являются востребованными в современной IT-индустрии, что обеспечивает профессиональную ценность приобретенных в ходе работы навыков.

Перспективы дальнейшего развития инфраструктуры включают: внедрение системы непрерывной интеграции и доставки (CI/CD) на базе GitLab CI или Jenkins; кластеризацию базы данных PostgreSQL для повышения отказоустойчивости; интеграцию автоматизированных тестов на Python с использованием фреймворков pytest и Selenium; настройку централизованного сбора логов с помощью стека ELK (Elasticsearch, Logstash, Kibana); организацию резервного копирования данных на внешний накопитель или в облачное хранилище.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Немет Э., Снайдер Г., Хейн Т.Р., Уэйли Б. UNIX и Linux: руководство системного администратора. 5-е изд. — М.: Вильямс, 2019. — 1184 с.: ил.

2. Олифер В.Г., Олифер Н.А. Компьютерные сети. Принципы, технологии, протоколы: Учебник для вузов. 5-е изд. — СПб.: Питер, 2020. — 1008 с.: ил.

3. Боуз Д. Docker. Сборник практических рецептов. — М.: ДМК Пресс, 2020. — 288 с.: ил.

4. Морозов Д.В. PostgreSQL 14. Практическое пособие по разработке и администрированию. — СПб.: БХВ-Петербург, 2022. — 512 с.: ил.

5. Савин А.И. Тестирование Дот Ком, или Пособие по жестокому обращению с багами в интернет-стартапах. — М.: ДМК Пресс, 2018. — 416 с.: ил.

6. Канер С., Фолк Д., Нгуен Б. Тестирование программ. — М.: Финансы и статистика, 2019. — 320 с.: ил.

7. Ромадина А.В. Ansible: создание сценариев для автоматизации задач ИТ-инфраструктуры. — М.: ДМК Пресс, 2021. — 352 с.: ил.

8. Карлсон К. Docker и Kubernetes: полное руководство. — СПб.: Питер, 2021. — 416 с.: ил.

9. Орлов С.А. Организация ЭВМ и систем: Учебник для вузов. 4-е изд. — СПб.: Питер, 2018. — 688 с.: ил.

10. Таненбаум Э., Уэзеролл Д. Компьютерные сети. 5-е изд. — СПб.: Питер, 2019. — 960 с.: ил.

11. Официальная документация Ubuntu Server [Электронный ресурс]. — URL: https://ubuntu.com/server/docs (Дата обращения: 20.06.2026).

12. Официальная документация Docker [Электронный ресурс]. — URL: https://docs.docker.com (Дата обращения: 20.06.2026).

13. Официальная документация PostgreSQL [Электронный ресурс]. — URL: https://www.postgresql.org/docs (Дата обращения: 20.06.2026).

14. Официальная документация Ansible [Электронный ресурс]. — URL: https://docs.ansible.com (Дата обращения: 20.06.2026).

15. Официальная документация OpenProject [Электронный ресурс]. — URL: https://www.openproject.org/docs (Дата обращения: 20.06.2026).

16. Официальная документация Netdata [Электронный ресурс]. — URL: https://learn.netdata.cloud (Дата обращения: 20.06.2026).

17. Официальная документация Hoppscotch [Электронный ресурс]. — URL: https://docs.hoppscotch.io (Дата обращения: 20.06.2026).

18. Официальная документация DBeaver [Электронный ресурс]. — URL: https://dbeaver.com/docs (Дата обращения: 20.06.2026).

19. Mozilla Firefox для разработчиков [Электронный ресурс]. — URL: https://www.mozilla.org/firefox/developer (Дата обращения: 20.06.2026).

20. Git — документация [Электронный ресурс]. — URL: https://git-scm.com/doc (Дата обращения: 20.06.2026).

21. Смирнов А.В. Виртуализация и облачные технологии: учебное пособие. — М.: ИНФРА-М, 2020. — 256 с.: ил.

22. Петров В.М. Администрирование Linux: практическое руководство. — СПб.: БХВ-Петербург, 2021. — 448 с.: ил.

23. Козлов Д.А. DevOps: практики непрерывной интеграции и доставки. — М.: ДМК Пресс, 2022. — 384 с.: ил.

24. ГОСТ Р 56939-2016. Защита информации. Разработка безопасного программного обеспечения. Общие требования. — М.: Стандартинформ, 2016.

25. ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering. — Geneva: ISO, 2018.